

CTOOD

Computational Thinking for object oriented design

LTPS - 3024

22SC1203

Syllabus :-

Session-1 :- Introduction to Course, introduction to object oriented programming

Session-2 :- Advantages of object oriented programming and key features.

Session-3 :- Primitive datatypes and arrays

Session-4 :- Building static methods.

Session-5 :- Modularization and Access Specifier

1) Private

2) Public

3) protected

4) default.

Session-6 :- Modularization at class level and package level

Session-7 :- if conditions and iterative statements.

Session-8 :- Wrapper classes and command line arguments.

Session-9 :- Method overloading

Session-10 :- classes objects and inner classes

Session-11 :- Accesses, mutators, two string method.

- Session - 12 :- Default Constructor, parametrised Constructor.
- Session - 13 :- Introduction to Java API - Scanner class
- Session - 14 :- Construct the chain, this - keyword Super - keyword.
- Session - 15 :- Aggregation and Composition
- Session - 16 :- Array list, Solid design principles
- Session - 17 :- Inheritance
- Session - 18 :- * Dynamic method dispatch
- Session - 19 :- Single turn pattern
- Session - 20 :- Abstract class and Abstract method.
- Session - 21 :- Final - keyword for method and class.
- Session - 22 :- Template pattern, Introduction to interface.
- Session - 23 :- Factory pattern
- Session - 24 :- * Generic classes, Applications of interface
- Session - 25 :- Java API for vector and linked list
- Session - 26 :- Comparable and Comparator, iterator interface.
- Session - 27 :- Anonymous class, decorative pattern

Session - 28 :- Event driven programming with event listeners (Swings)

Session - 29 :- Event driven programming

Session - 30 :- Exceptions

Session 31 :- Java API for Exceptions

Session 32 :- Try, Catch, Finally

Session 33 :- Throw, and Throws - keywords

Session 34 :- User defined exceptions

Session 35 :- Files

Session 36 :- Serializable

Session 37 :- object IO using Serializable

Session 38 :- String, string tokenizer and String builder

Session 39 :- String constant pool.

Text books :-

1) "Complete Reference for JAVA" - 9th Edition
TMH

2) "Cay.s. Horstmann and Gary Cornell", "Core Java 2, Volume 1".

3) Introduction to JAVA - By bala guru Swamy

Downloads :-

→ Java SE download

i) jdk 19 download for windows

ii) Eclipse 2021 - 22.

Note :

- 1) Inside the class, we have to write data members and member functions include main method.
- 2) Outside the class we should not write a single statement.

Process :-

- 1) Create a Project
- 2) Create a package
- 3) Create the classes.

Syntax for class :-

```
public class Name
{
    Data member
    member functions
    Main() { }
}
```

System.out.println(); → to print message

Naming Convention for class is Every word initial letter must be Capital

Eg:- Hello World

System

First

Naming Convention for method except first word remaining word initial must be Capital

Naming Convention for package is all letters should be Small.

Naming Convention for Constant is all letters should be Capital letters.

Eg:- Final int PI = 3.14;

Q) write a Java program to print "Hello world".

Sol:-

```
package s37;  
public class program1  
{  
    public static void main (String[] args)  
    {  
        System.out.println("Hello world");  
    }  
}
```

Q) write a Java to perform the Sum of two numbers.

Sol:-

```
package s37;  
public class Addition  
{  
    public static void main (String[] args)  
    {  
        Addition sum = new Addition();  
        sum.add(10, 20);  
    }  
}
```

```

public void add (int a ,int b)
{
    int res ;
    res = a+b ;
    System.out.println ("Result is " + res);
}
}

```

Note :-

→ we are creating an object in order to call the function.

→ we create the objects for the classes.

Q) write the java program which creates the calculator, case-1 : Addition, Subtraction, multiplication, division using Switch statement

In order to get the data from keyboard we have Scanner class it is available in util package.

```

int x ;
Scanner sc = new Scanner (System.in);
x = sc.nextInt();

```

Methods :-

nextInt ();

nextDouble ();

nextByte ();

nextFloat ();

next ();

nextLine ();

nextChar ();

Program for Scanner function.

```
package pl;  
import java.util.*;  
public class Addition  
{  
    public static void main(String[] args)  
    {  
        Addition ob1 = new Addition();  
        int a,b;  
        Scanner sc = new Scanner(System.in);  
        a = sc.nextInt();  
        b = sc.nextInt();  
        ob1.add(a,b);  
        ob1.Sub(a,b);  
    }  
    public void add(int a,int b)  
    {  
        System.out.println("Add is "+(a+b));  
    }  
    public void Sub(int a,int b)  
    {  
        System.out.print("Sub is "+(a-b));  
    }  
}
```

Program for Calculator.

Sol:

```
package s37;
```

```
import java.util.*;
```

```
Public class Calculator
```

```
{
```

```
public static void main (String[] args)
```

```
{
```

```
Calculator obj1 = new Calculator();
```

```
int a,b,ch;
```

```
Scanner sc = new Scanner(System.in);
```

```
a = sc.nextInt();
```

```
b = sc.nextInt();
```

```
ch = sc.nextInt();
```

```
obj1.calculate(a,b,ch);
```

```
}
```

```
public void calculate(int a,int b,int ch)
```

```
{
```

```
switch(ch)
```

```
{
```

```
Case 1 :
```

```
System.out.println(a+b);
```

```
break;
```

```
Case 2 :
```

```
System.out.println(a-b);
```

```
break;
```

```
Case 3 :
```

```
System.out.println(a*b);
```

```
break;
```

Case 4 :

```
System.out.println(a/b);
```

default :

```
System.out.println("Enter valid number");
```

```
}
```

```
}
```

```
}
```

Q) write the java program which find out the Simple interest.

Sol:

```
package S37;  
import java.util.*;  
public class SimpleInterest  
{  
    public static void main (String [] args)  
    {  
        SimpleInterest obj1 = new SimpleInterest()  
        float p,t,r;  
        Scanner sc = new Scanner (System.in);  
        p = sc.nextFloat();  
        t = sc.nextFloat();  
        r = sc.nextFloat();  
        System.out.println("Result is "+obj1.SI(p,t,r);  
    }  
    public float SI (float p, float t, float r)  
    {  
        float res;  
        res = (p*t*r)/100;  
    } return res;  
}
```

Q) write the java program which find out the factorial of a number.

Sol:-

```
package S37 ;  
import java.util.* ;  
public class Factorial  
{  
    public static void main (String[] args)  
    {  
        Factorial obj1 = new Factorial ( ) ;  
        int n ;  
        Scanner sc = new Scanner (System.in) ;  
        n = sc.nextInt ( ) ;  
        System.out.println (obj1.fact (n)) ;  
    }  
    public int fact (int n)  
    {  
        int res = 1 ;  
        for (int i = 1 ; i <= n ; i++)  
        {  
            res = res * i ;  
        }  
        return res ;  
    }  
}
```

8) write the java program which find out the sum of digits in a given number.

Sol:-

```
package s37;  
import java.util.*;  
public class SumOfDigits  
{  
    public static void main (String[] args)  
    {  
        SumOfDigits obj1 = new SumOfDigits();  
        int n;  
        Scanner sc = new Scanner (System.in);  
        n = sc.nextInt();  
        System.out.println(obj1.sum(n));  
    }  
    public int sum(int n)  
    {  
        int x, Sum = 0;  
        while (n != 0)  
        {  
            x = n % 10;  
            Sum = Sum + x;  
            n = n / 10;  
        }  
        return Sum;  
    }  
}
```

Model 2 :

A File can contain a no. of classes but a class which contains the main method has to declare public and has to save with that class name only.

Eg :-

```
package p1 ;  
class Fact1  
{  
    public long fact (int n)  
    {  
        long f = 1 ;  
        int i ;  
        for (i = 1 ; i <= n ; i++)  
        {  
            f = f * i ;  
        }  
        return f ;  
    }  
}  
  
public class Demo  
{  
    public static void main (String[] args)  
    {  
        Fact1 a1 = new Fact1 (.) ;  
        System.out.println (a1.fact (4)) ;  
    }  
}
```

Model 3 :

Model 3 is called within the package level. Here 2 different classes are created. one should be named as Demo (which contains main method).

Eg: Demo2

```
package p1 ;

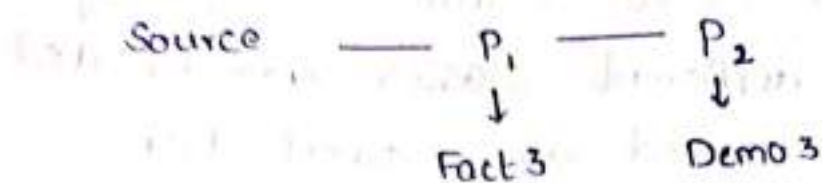
public class Demo2
{
    public static void main (String args[])
    {
        Fact2 a1 = new Fact2 ();
        System.out.println (a1. fact (4));
    }
}
```

Fact2

```
package p1 ;

public class Fact2
{
    public int fact (int n)
    {
        int f = 1 ;
        for (int i = 1 ; i <= n ; i++)
        {
            f = f * i ;
        }
        return f ;
    }
}
```

Model-4 :- It is package level.



It is mandatory to write an import statement
i.e. `import P1.Fact3;`

Example: P1 - Fact3 :-

```
package P1;
public class Fact3
{
    public int fact(int n)
    {
        int f = 1;
        for(int i = 1; i <= n; i++)
        {
            f = f * i;
        }
        return f;
    }
}
```

P2 - Demo3 :-

```
package P2;
import P1.Fact3;
public class Demo3
{
    public static void main(String args[])
    {
        Fact3 a1 = new Fact3();
        System.out.println(a1.fact(4));
    }
}
```

Note :-

Model 1 — class level

Model 2 and Model 3 — within package level

Model 4 — Package level.

Problems :-

① Sum of digits

② Simple interest

③ Prime number or not

④ Fibonacci Series

⑤ Palindrome

Above problems do it in model 1, 3, 4.

Solutions

① SumOfDigits :-

Model-1 :-

```
package a1;  
import java.util.*;  
public class SumOfDigits  
{  
    public static void main (String[] args)  
    {  
        SumOfDigits obj1 = new SumOfDigits();  
        int n;  
        Scanner sc = new Scanner(System.in);  
        n = sc.nextInt();  
        sc.close();  
        System.out.println(obj1.add(n));  
    }  
}
```

```
public int add(int n)
```

```
{
```

```
    int res = 0, r;
```

```
    while (n != 0)
```

```
    {
```

```
        r = n % 10;
```

```
        res = res + r;
```

```
        n = n / 10;
```

```
    }
```

```
    return res;
```

```
}
```

```
}
```

Model-2 :-

```
package a1;
```

```
import java.util.*;
```

```
class Sum
```

```
{
```

```
    public int add(int n)
```

```
    {
```

```
        int res = 0, r;
```

```
        while (n != 0)
```

```
        {
```

```
            r = n % 10;
```

```
            res = res + r;
```

```
            n = n / 10;
```

```
        }
```

```
        return res;
```

```
    }
```

```
}
```

public class SumOfDigits

```
{
    public static void main (String[] args)
    {
        Sum obj1 = new Sum();
        int n;
        Scanner sc = new Scanner(System.in);
        n = sc.nextInt();
        sc.close();
        System.out.println(obj1.add(n));
    }
}
```

Model-3 :-

Demo1

```
package a1;
import java.util.*;
public class Demo1
{
    public static void main (String[] args)
    {
        SumOfDigits obj1 = new SumOfDigits();
        int n;
        Scanner sc = new Scanner(System.in);
        n = sc.nextInt();
        sc.close();
        System.out.println(obj1.add(n));
    }
}
```

SumOfDigits :

```
package a1;  
public class SumOfDigits  
{  
    public int add(int n)  
    {  
        int r, res = 0;  
        while(n != 0)  
        {  
            r = n % 10;  
            res = res + r;  
            n = n / 10;  
        }  
        return res;  
    }  
}
```

Model-4 :-

Demo1 :-

```
package a2;  
import java.util.*;  
import a1. SumOfDigits;  
public class Demo1  
{  
    public static void main(String[] args)  
    {  
        SumOfDigits obj1 = new SumOfDigits();  
        int n;
```

```

Scanner sc = new Scanner (System.in);
n = sc.nextInt();
sc.close();
System.out.println (obj1.add(n));
}
}

```

SumOf Digits :-

```

package a1;
public class SumOf Digits

```

```

{
    public int add (int n)
    {
        int r, res = 0;
        while (n != 0)
        {
            r = (n % 10);
            res = res + r;
            n = n / 10;
        }
        return res;
    }
}

```

```

}
}

```

Q) Write a program to read n value and Print the multiplication table till 10.

```
sol: package pl;
import java.util.*;
public class Table
{
    public static void main (String[] args)
    {
        Table obj1 = new Table();
        int n;
        Scanner sc = new Scanner(System.in);
        n = sc.nextInt();
        int i = 1;
        obj1.table(n);
    }
    public void table (int n)
    {
        for (i = 1; i <= 10; i++)
        {
            System.out.println(n + " x " + i + " = " + (n*i))
        }
    }
}
```

output :-

5 x 1 = 5
5 x 2 = 10
5 x 3 = 15

Input and Output Strings :-

- 1) System.in → Reads Data from keyboard.
- 2) System.out → Displays data on screen.
- 3) System.err → Display error message

* Into a variable we can store a value in 4 ways

- i) Hardcore - Assign
- ii) Scanner - Read
- iii) CommandLine Arguments — Runtime
- iv) Files - Files.

i) @ write a program to add two numbers using hardcore values.

Sol:-

```
package p1;  
public class Add  
{  
    public static void main (String Args[])  
    {  
        int a = 10, b = 20, Sum;  
        Sum = a + b;  
        System.out.print (Sum);  
    }  
}
```

Output :- 30

Q) Addition of two numbers using Scanner function.

Sol:-

```
package pl;  
import java.util.Scanner;  
public class add  
{  
    public static void main (String args[])  
    {  
        Scanner input = new Scanner (System.in);  
        int a = input.nextInt();  
        int b = input.nextInt();  
        int sum = a+b;  
        System.out.print (sum);  
        input.close();  
    }  
}
```

output:- 10
20
30.

Static :-

static can be applied as

- ① as Method
- ② Data member
- ③ static blocks

Static block :-

Generally, the compiler will execute from the main method. If you have written static block, the compiler will execute the static block first and then from the main method.

In the static block, we will initialize the static variables and also we can perform some operations also.

Syntax for static block :-

```
static
{
    Code;
}
```

Q) write a java program which demonstrate the static block.

Sol.

```
public class A
{
    public static void main (String[] args)
    {
        System.out.println("Main Method");
    }
}
```

Static

```
{  
    System.out.println("Static block");  
}
```

output : Static block
Main method

Q) Can you write java program without main method?

sol: Yes, it is possible in less versions of java 1.6. If it is more than Java 1.6 it is not possible.

Java 1.6 onwards, they put constraint that these should be main method.

Static method:-

A static method will be called by the class name. A static method can access only static data and static method.

Eg:- A static method can be called without an objective object.

Q) write a java program which performs a addition operation using static method.

sol:-
package p1;

public class Add
{

```

public static void Addition(int a, int b)
{
    System.out.println(a+b);
}

public static void main (String[] args)
{
    addition(2,3);
}

```

output : 5.

Q) why the main method is static?

sol:- The main method is called by the compiler using the class name.

static variable :-

- 1) The initial value of the static variable is 0
- 2) Static variables are used inside the static method only.
- 3) Only one copy of memory is created.
- 4) Static variables are called class variables
- 5) Static variables are shared among the different objects.

Eg:-

```

package p1;

public class Add
{
    public static int a;
    public static int b;
}

```

```

static int c;
static int c;
{
    a = 2;
    b = 3;
}
public static void Addition()
{
    System.out.println(a+b);
}
public static void main (String[] args)
{
    Addition();
}
}

```

Output : 5

In the static method, you cannot access the normal variables.

Q) write a java program which find the factor of a given number using n as a data member.

sol:-

```

package p1;
import java.util. Scanner ;
public class Fact
{
    public int n;
    public void factorial (-)
    {

```

```

long f = 1;
for (int i = 1; i <= n; i++)
{
    f = f * i;
}
System.out.println(f);
}

public static void main (String[] args)
{
    Scanner sc = new Scanner (System.in);
    Fact ob1 = new Fact ();
    ob1.n = sc.nextInt();
    ob1.factorial ();
}

```

Data members can be accessed in the member functions. Directly or through the object.

Q) Write a java program which performs the addition of 3 numbers where a, b, c is a data members.

Sol:-

```

package p1;
import java.util.Scanner;
public class Add
{
    public int a, b, c;
    public void addition ()
    {
        System.out.println (a+b+c);
    }
    public static void main (String[] args)
    {

```

```
Scanner sc = new Scanner(System.in);
```

```
Add obj1 = new Add();
```

```
obj1.a = sc.nextInt();
```

```
obj1.b = sc.nextInt();
```

```
obj1.c = sc.nextInt();
```

```
obj1.addition();
```

```
}
```

Example :-

```
package p1;
```

```
// import java.util.Scanner;
```

```
public class Add
```

```
{
```

```
    public int a;
```

```
    public int b;
```

```
    public int c;
```

```
    public static int d;
```

```
    public void addition()
```

```
{
```

```
        System.out.println(a);
```

```
        System.out.println(b);
```

```
        System.out.println(c);
```

```
        System.out.println(d);
```

```
}
```

```
    public static void main(String[] args)
```

```
{
```

```
        Add obj1 = new Add();
```

```
ob1. a = 2 ;  
ob1. b = 3 ;  
ob1. c = 4 ;  
ob1. d = 7 ;
```

```
ob1.addition ( ) ;
```

```
ob2. a = 4 ;
```

```
ob2. b = 5 ;
```

```
ob2. c = 6 ;
```

```
ob2.addition ( ) ;  
}
```

Two String method :-

Two String is a predefined method, which is defined in the Object class.

Every class which is defined by the user is extend by Object class.

whenever you try print System.out.println(ob1), it automatically calls the two String() method.

If you didnot write the two String() method then it will print the object reference.

The syntax for the two String method is

```
public String toString ( )  
{  
  
}
```

Example:-

```
package pl;  
import java.util.Scanner;  
public class toStringex  
{  
    public int id;  
    public String name;  
    public void getvalues()  
    {  
        Scanner sc = new Scanner(System.in);  
        name = sc.nextLine();  
        id = sc.nextInt();  
    }  
    public String toString()  
    {  
        return id + " " + name;  
    }  
    public static void main (String[] args)  
    {  
        toStringex ob1 = new toStringex();  
        ob1.getvalues();  
        System.out.println(ob1);  
    }  
}
```

In java we have a default package, no need to import, name is `java.lang.*`;
→ `Math` is a function in this.

Eg:-

```
System.out.println(Math.pow(5,2));  
System.out.println(Math.sqrt(4));
```

For any program use these 3 functions

① `get values()`

② `logic()`

③ `print values()`

Eg:-

```
package p1;
```

```
public class toString
```

```
{ public String name;  
  public void getValues()
```

```
{  
    name = "ram";  
}
```

```
public void printValues()
```

```
{  
    System.out.println(name);  
}
```

```
public static void main (String[] args)
```

```
{  
    toString ob1 = new toString();  
    ob1.getValues();  
    ob1.printValues();  
}
```

Q) write a java program to find the roots of the Quadratic Equation. Use `getvalues()`, `logic` and `printvalues()` to read and print.

sol:-

```
package p1;  
import java.util.Scanner;  
public class Roots  
{  
    public int a,b,c;  
    public double r1,r2;  
    public void getvalues()  
    {  
        Scanner sc = new Scanner(System.in);  
        a = sc.nextInt();  
        b = sc.nextInt();  
        c = sc.nextInt();  
        sc.close();  
    }  
    public void logic()  
    {  
         $r1 = ((-b) + (\text{Math.sqrt}(\text{pow}(b,2) - (4*a*c)))) / 2*a;$   
         $r2 = ((-b) - (\text{Math.sqrt}(\text{pow}(b,2) - (4*a*c)))) / 2*a;$   
    }  
    public void printvalues()  
    {  
        System.out.print("Roots are");  
        System.out.println(r1 + " " + r2);  
    }  
}
```

```

public static void main (String[] args)
{
    Roots ob1 = new Roots();
    ob1.getvalues();
    ob1.logic();
    ob1.printvalues();
}
}

```

output:-

1 5 6
 Roots are -3.00 -2.00.

Q) write a java program of class Students which contains the details like name, id, mail, phno and marks of 3 subject. Now you have to display and find percentage.

Sol:-

```

package p1;
import java.util.Scanner;
public class Students
{
    public String name, email;
    public long id, phno;
    public static int sec, total;
    public int M1, M2, M3;
    public float p;
    static
}

```

```

    Sec = 37;
    total = 300;
}
Scanner sc = new Scanner(System.in);
public void getvalues()
{
    name = sc.next();
    email = sc.next();
    id = sc.nextLong();
    phno = sc.nextInt();
}
public void logic()
{
    P = ((M1 + M2 + M3) / total) * 100;
}
public void printvalues()
{
    System.out.println("name + " + id);
    System.out.println("phno + " + email);
    System.out.println("Sec + " + P);
}
public static void main (String[] args)
{
    for (int i; i < 4; i++)
    {
        Students obj = new Students();
        obj.getvalues();
        obj.percentage();
        obj.printvalues();
    }
}
}

```

Function Overloading:

It is one of the feature in java programming with same name and different functionality. This concept is not there in the c language.

Polymorphism → under one name with different functionality.

Function overloading will depends on number of arguments as well as type of argument.

The Function overloading does not depends on

the return type.

Eg:-

add ();

add (int a, int b);

add (int a);

add (int a, int b, int c);

add (float a, float b)

(↑) These are accepted

int add (int a, int b) }
float add (int x, int y) } wrong
process.

polymorphism:

Under the single name we have different behaviours.

It is of two types:

① Function overloading.

② Function overriding.

Java doesnot support operator overloading.

Internally java supports operator overloading but it will never allow to do operator overloading.

Q) write a java program demonstrate the function overloading?

Sol:

```
package P1;
public class funcover()
{
    public void add()
    {
        int a = 2, b = 3;
        System.out.println(a+b);
    }
    public void add(int a)
    {
        int b = 3;
        System.out.println(a+b);
    }
    public void add(int a, int b)
    {
        System.out.println(a+b);
    }
    public static void main(String[] args)
    {
        funcover obj = new funcover();
        add();
        add(2);
        add(2,3);
    }
}
```

Constructors :-

It is a special method in the class. The purpose of constructor is in order to initialize the data members.

Rules for

- ① The name of the constructor is same as class name.
 - ② No return type (even void). It is called automatically whenever object is created.
 - ③ You should not write static, final, synchronized for constructor.
 - ④ We have a constructor overloading and we have a constructor chaining.
- Q) write a java program which creates the class name as Student and data members are name, Id and use zero argument constructor to initialize the data and use toString() method to print the student details.

```
sdr package p1;  
import java.util.Scanner;  
public class Student  
{  
    public int id;  
    public String name;  
    Student()  
    {  
        Scanner sc = new Scanner(System.in);  
        name = sc.nextLine();  
    }  
    id = sc.nextInt();  
}
```

```
public String toString()
```

```
{  
    return name + " " + id;  
}
```

```
public static void main (String[] args)
```

```
{  
    student obj = new student();  
    System.out.println(obj);  
}
```

A Zero argument Constructor is a constructor where no argument are there and user has written that constructor.

Here getvalues() is done by the constructor. and printvalues() work done by the toString().

Constructors are two types:-

① default Constructor

② parameterise constructor.

Default Constructor means a zero argument constructor which is provided by the Compiler.

Parametrized Constructor :

If the constructor has arguments then it is parametrized constructor.

Eg for Constructor overloading & parametrized :-

```
package p1;
import java.util.Scanner;
public class Student
{
    public int id1;
    public String name;
    Student ()
    {
        id1 = 0;
        name = " ";
    }
    Student (int id2)
    {
        id1 = id2;
        name = " kumar ";
    }
    Student (int id3, String name1)
    {
        id1 = id3;
        name = name1;
    }
    public String toString()
    {
        return name + " " + id;
    }
}
```

```
public static void main (String[] args)
```

```
{  
    Student s1 = new Student ();  
    Student s2 = new Student (95);  
    Student s3 = new Student (96, "phanindra");  
    System.out.println (s1);  
    System.out.println (s2);  
    System.out.println (s3);  
}  
}
```

this keyword:-

- 1) The this keyword always refers the current object.
- 2) Using this keyword you can call the constructor chaining.
- 3) this keyword always refers the data member

Example of this keyword:-

- Q) Define two data members a, b. write the getvalues() (8) 2 argument constructor and assign the values

sol:-

```
package p1;  
public class thiseg()  
{  
    public int a;  
    public int b;
```

```
thiseg (int a, int b)
```

```
{  
    this.a = a;  
    this.b = b;  
}  
public String toString()  
{  
    return a + " " + b;  
}  
public static void main (String[] args)  
{  
    thingeg ob1 = new thiseg (2, 3);  
    System.out.println (ob1);  
}  
}
```

output :- 2 3

Constructor chaining :-

Eg :-

```
package P1;  
public class chaining  
{  
    public int a;  
    public int b;  
    public int c;  
    public chaining()  
    {  
        a = 0;  
        b = 0;  
        c = 0;  
    }  
}
```

```
chaining (int x)
```

```
{  
    a = x;  
}
```

```
chaining (int x, int y)
```

```
{  
    this(x);  
    b = y;  
}
```

```
chaining (int x, int y, int z)
```

```
{  
    this(x, y);  
    c = z;  
}
```

```
public static void main (String[] args)
```

```
{  
    chaining ob1 = new chaining (2, 3, 4);
```

```
    System.out.println (ob1);
```

```
    chaining ob2 = new chaining ();
```

```
    System.out.println (ob2);
```

```
    public String toString ()
```

```
    {  
        return a + " " + b + " " + c;
```

```
    }
```

```
}
```

output :- 2 3 4
 0 0 0

Command line arguments :-

In order to convert from string to integer

```
int a = Integer.parseInt("200");
```

Command line argument means, if you pass the string values to the args variable of the main function.

Then we say that, command line argument.

Q) write a java program which pass the 10, 20, 30 and perform the addition using command line argument.

```
Sol:- package p1;
public class Comex
{
    public static void main (String[] args)
    {
        int a, b, c, d, sum;
        a = Integer.parseInt (args[0]);
        b = Integer.parseInt (args[1]);
        c = Integer.parseInt (args[2]);
        d = Integer.parseInt (args[3]);
        sum = a+b+c+d;
```

```
        System.out.println (sum);
    }
}
```

click on file name, so to run as then go to arguments then given values 10 20 30 40, then click apply and run.

Q) write the above program using for loop.

Sol: package p1;

public class comex

{
public static void main (String[] args)

{
int sum = 0;

for (int i = 0; i < args.length; i++)

{
sum = sum + Integer.parseInt(args[i]);

System.out.println (sum);

}

output :- 100.

Q) write a java program which reads four names from command line arguments and print it.

Sol: package p1;

public class comex

{

public static void main (String[] args)

{
for (int i = 0; i < args.length; i++)
System.out.println (args[i]);

}

}

Q) write a java program which reads five values from command line arguments and print maximum and minimum terms.

```
sol:- package p1;
public class check
{
    public static void main (String[] args)
    {
        int max = Integer.parseInt (args[0]);
        int min = Integer.parseInt (args[0]);
        for (int i = 0; i < args.length; i++)
        {
            if (Integer.parseInt (args[i]) > max)
            {
                max = Integer.parseInt (args[i]);
            }
        }
        for (int i = 0; i < args.length; i++)
        {
            if (Integer.parseInt (args[i]) < min)
            {
                min = Integer.parseInt (args[i]);
            }
        }
        System.out.println ("Maximum is " + max);
        System.out.println ("Minimum is " + min);
    }
}
```

wrapper classes:-

1. Auto boxing $\rightarrow$ primitive (int) to class
2. Unboxing $\rightarrow$ class to primitive (int).

primitive

int
float
double
char
long

class

Integer.
float
double
character
long

For auto boxing we use a method called `valueOf()`

For unboxing we use a method called `datatypeValue()`.

Example:-

```
package p1;  
public class Wrapper {  
    public static void main (String[] args)  
    {  
        int x = 5;  
        Integer y = Integer.valueOf(x);  
        System.out.println(y);  
        int z = y.intValue();  
        System.out.println(z);  
    }  
}
```

output :- 5
5

Copy Constructor :-

Creating a clone of one object is called Copy Constructor.

In the below example you created a object ob1 with data member values 2,3,4. Then we write a function on constructor to clone the ob1. So that, we can create ob2 object with same data values.

Q) write an example which explains the Copy Constructor.

Sol:

```
package p1;

public class Copycon
{
    public int a,b,c;

    Copycon (int x, int y, int z)
    {
        a = x;
        b = y;
        c = z;
    }

    public String toString ()
    {
        return a + " " + b + " " + c;
    }

    Copycon (Copycon ob1)
    {
        a = ob1.a;
        b = ob1.b;
```

```
c = ob1.c ;  
}
```

```
public static void main (String[] args) {  
    Copycon ob1 = new Copycon (2,3,4);  
    System.out.println (ob1);  
    Copycon ob2 = new Copycon (ob1);  
    System.out.print (ob2);  
}
```

when we are cloning object using constructor then we call it as copy constructor.

Q) write a javaprogram which creates the Complex class. the data members are a, b and calculate the addition of 2 numbers and subtraction of 2 complex numbers and multiplication of 2 complex numbers.

Sol:

```
package p1;  
public class Complex  
{  
    public int a, b;  
    Complex (int x, int y)  
    {  
        a = x;  
        b = y;  
    }  
    public void add (Complex c2)  
    {  
        int r1, r2;
```

```
    r1 = a + c2.a;  
    r2 = b + c2.b;  
    System.out.println(r1 + " + " + r2 + "i");
```

```
}  
public void Sub (Complex c2)
```

```
{  
    int r1, r2;  
    r1 = a - c2.a;  
    r2 = b - c2.b;
```

```
    System.out.println(r1 + " + " + r2 + "i");
```

```
}  
public void Mul (Complex c2)
```

```
{  
    int r1, r2, r3, r4;  
    r1 = a * c2.a;  
    r2 = a * c2.b;  
    r3 = b * c2.a;  
    r4 = b * c2.b;
```

```
    System.out.println(r1 - r4 + " + " + (r2 + r3) + "i")
```

```
}  
public static void main (String[] args)
```

```
{
```

```
    Complex c1 = new Complex (2, 3);
```

```
    Complex c2 = new Complex (4, 6);
```

```
    c1.add(c2);
```

```
    c1.Sub(c2);
```

```
    c1.Mul(c2);
```

```
}
```

```
}
```

output :-

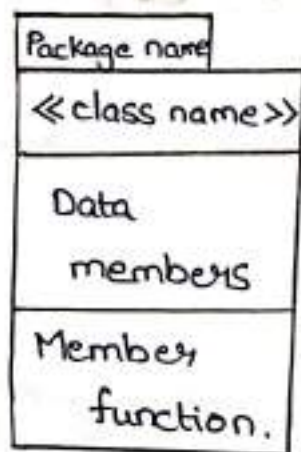
$$\begin{aligned} &6 + (9i) \\ &-2 + (-3i) \\ &-10 + (24i) \end{aligned}$$

class diagram :-

class diagram is one of the diagram for set of UML diagram.

* The pictorial representation of the java class is called class diagram.

Eg :-



* We have 4 access specifiers. They are

- i) private -
- ii) public +
- iii) default ~
- iv) Protected #

* 1) static method or data member → underline.

2) Final keyword should be in capital letters

3) The class diagram will outline the only the methods as well as data members and their return types and no. of arguments and their type. It doesn't bother about the

code inside the member function.

Eg:-

```
package p1;
```

```
class A
```

```
{
```

```
    int a;
```

```
    public int b;
```

```
    protected String name;
```

```
    private long phno;
```

```
}
```

```
    A()
```

```
{
```

```
        //
```

```
}
```

```
    public void getValues(a, b, name, phno)
```

```
{
```

```
        //
```

```
}
```

```
    public void printValues()
```

```
{
```

```
        //
```

```
}
```

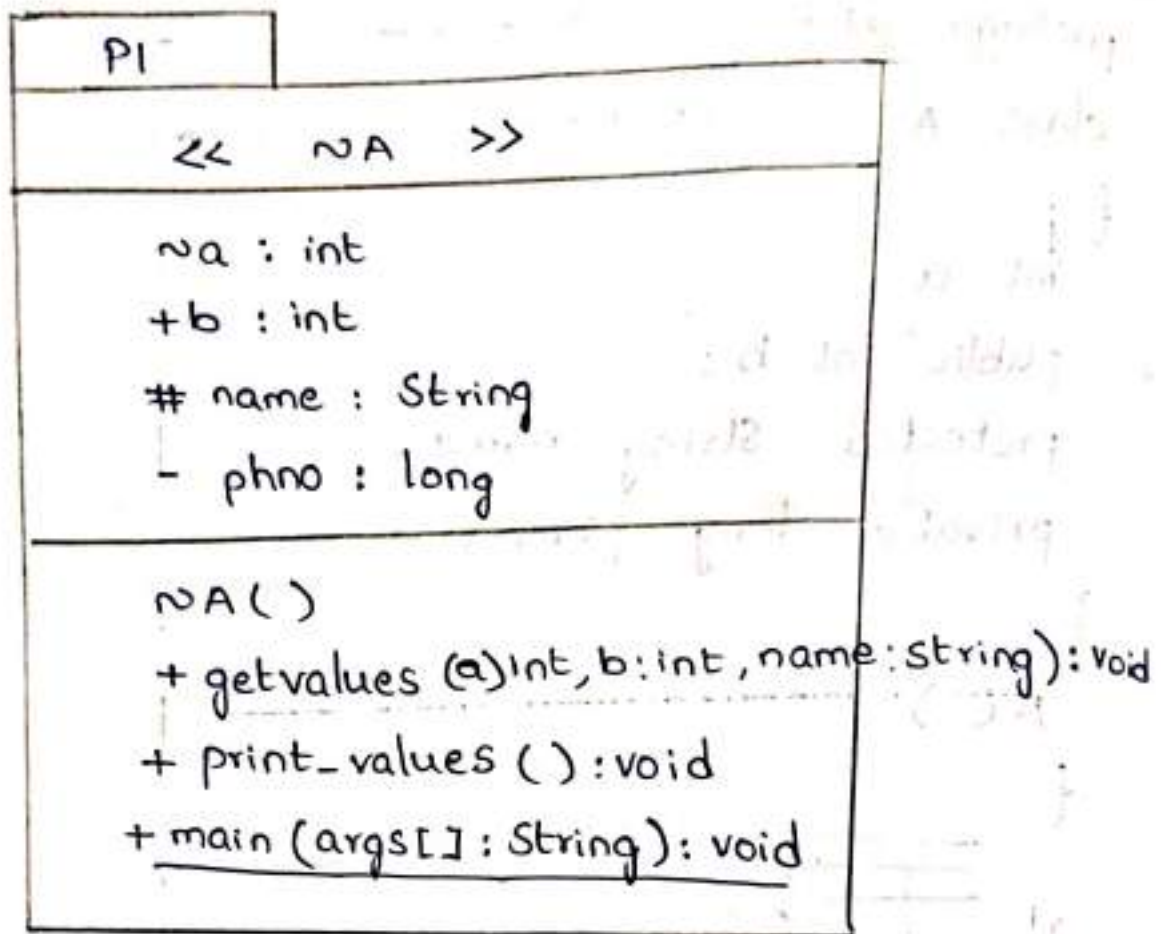
```
    public static void main(String[] args)
```

```
{
```

```
        //
```

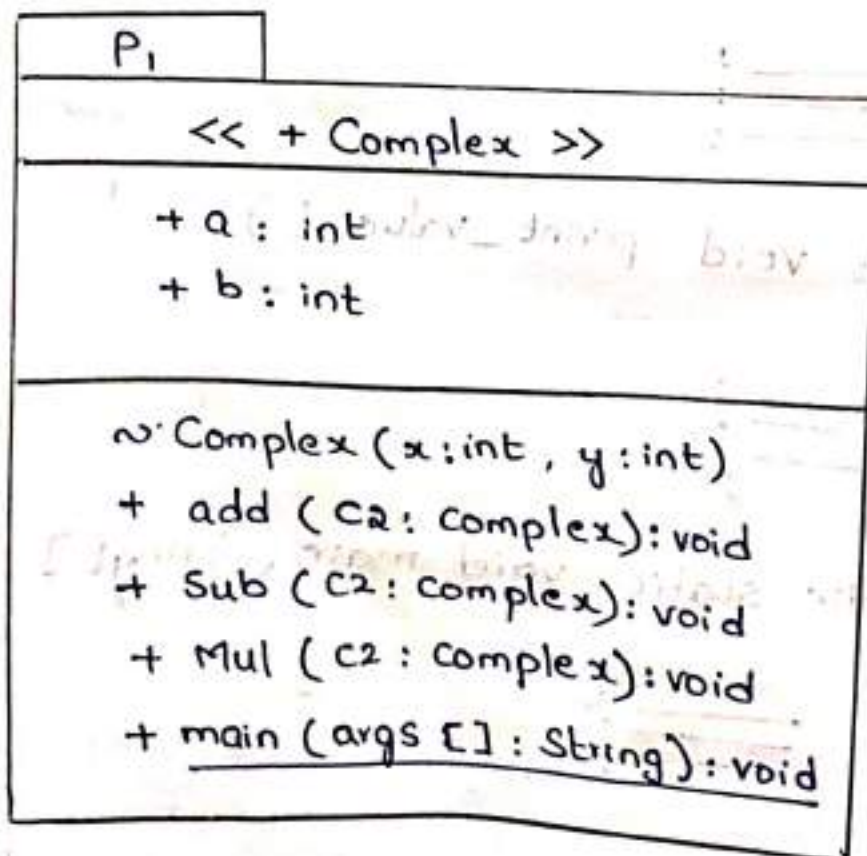
```
}
```

class diagram :-



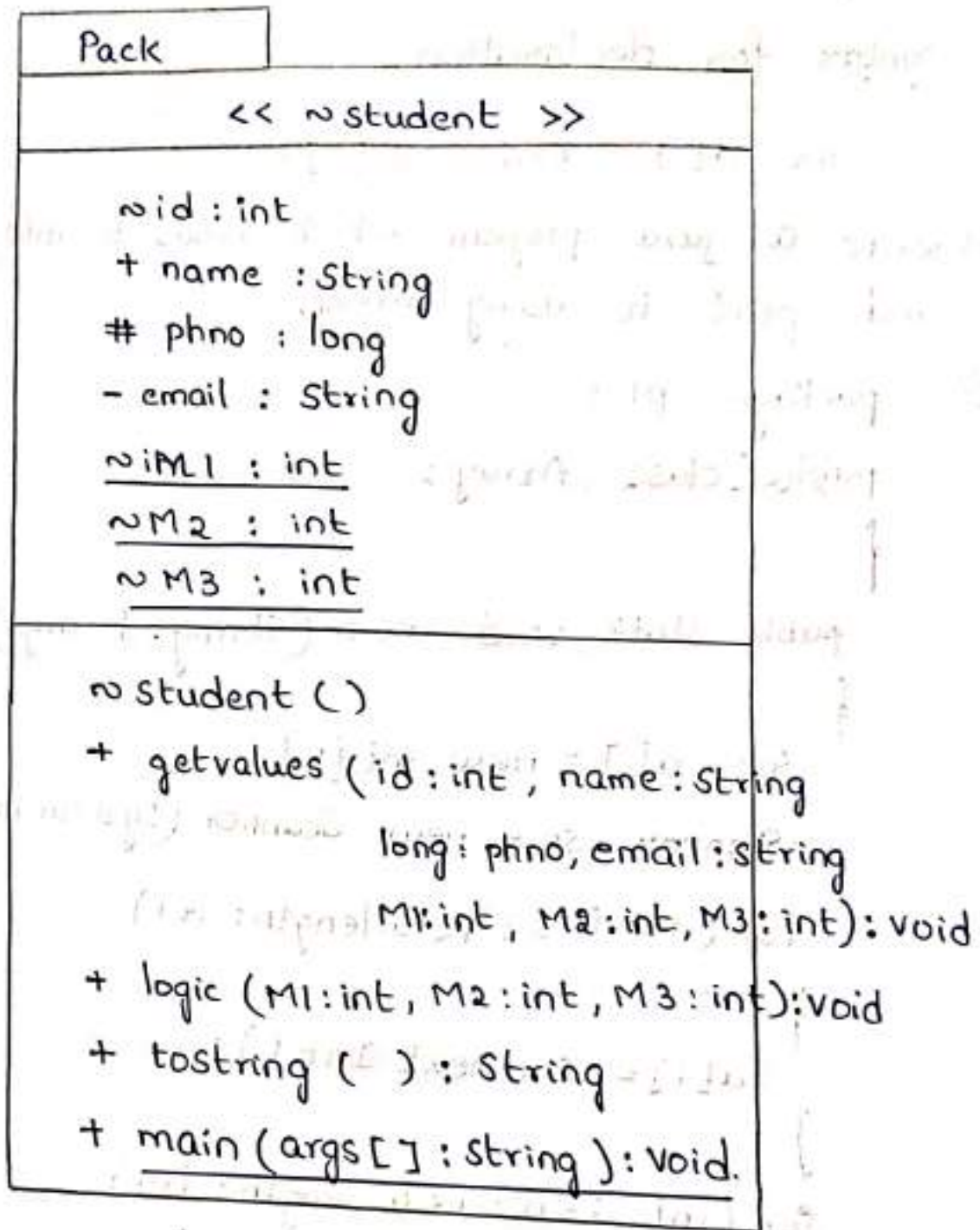
Q) Draw the class diagram for the Complex numbers.

Sol:-



a) Draw the class diagram for the student sum.

solt:-



Arrays in Java :-

Syntax for declaration :-

```
int a[] = new int[5];
```

Q) write a java program which reads 5 integers and print it using array.

sol:

```
package pl;
```

```
public class Array;
```

```
{
```

```
    public static void main (String[] args)
```

```
    {
```

```
        int a[] = new int[5];
```

```
        Scanner sc = new Scanner (System.in);
```

```
        for (int i=0; i<a.length; i++)
```

```
        {
```

```
            a[i] = sc.nextInt();
```

```
        }
```

```
        for (int i=0; i<a.length; i++)
```

```
        {
```

```
            System.out.println(a[i]);
```

```
        }
```

```
    }
```

```
}
```

output:- 9 21 36 92 54

9 21 36 92 54.

short form of for loop:-

```
for (int k : a)
{
    System.out.print(k + " ");
}
```

↪ array name

For initialization of array:-

```
int a[] = new int[] {1, 2, 3, 4, 5};
```

Array of objects:-

Q) write a student class where the data members are id, name

i) write the 2 argument constructor to initialize data member.

ii) to print data members use toString method.

iii) define 2 students using array of objects.

Sol:- package p1;

```
public class Studentarr
```

```
{
```

```
    int id;
```

```
    String name;
```

```
    Studentarr (int id1, String name1)
```

```
{
```

```
        id = id1;
```

```
        name = name1;
```

```
}
```

```

public String toString ()
{
    return id + " " + name + " ";
}

public static void main (String[] args)
{
    Studentarr s[] = new Studentarr[2];
    s[0] = new Studentarr (100, "ram");
    s[1] = new Studentarr (200, "naveen");
    for (int i=0; i < s.length; i++)
    {
        System.out.println (s[i]);
    }
}

```

output :- 100 ram
200 naveen.

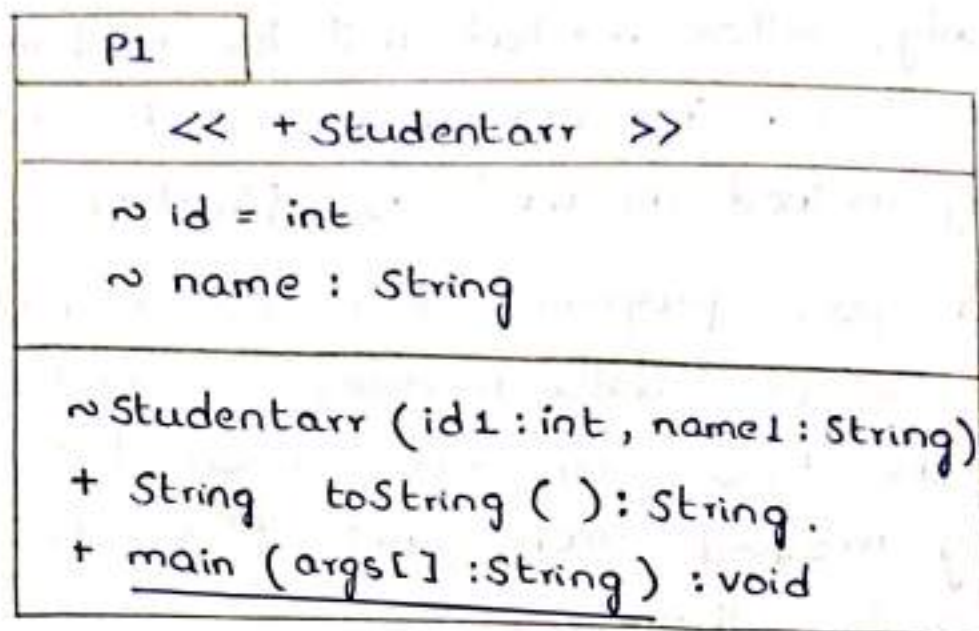
Syntax for Array of objects :-

```

classname a[] = new classname [100];
a[0] = new classname (Datamembers);
a[1] = new classname (Datamembers);
⋮
a[99] = new classname (Datamembers);

```

class diagram :-



Setters and getters / Mutators and accessors :-

- 1) If we have a two data members then we will have 2 setter methods and 2 getter methods.
- 2) The purpose of setter method is to set the value of data member.
- 3) The purpose of getter method is to get the value of data member.
- 4) All the setter and getter methods should be public.
- 5) Setter method will have arguments but no return type.
- 6) getter method will not have an argument but will have a return type.

Eg:- Data members are id, name

```
public void setId (int id1)
public void setName (String name1)
public int getId ( )
```

```
public String getName ( )
```

7) Generally, setter method will be used in the constructor. Getter method will be used in toString method as well as function.

Q) Write a java program which create a class called Student, data members are id, name. Create the two argument constructor and toString method and print details using setters and getters.

sol:

```
package p1;
public class Student
{
    public int id;
    public String name;

    Student (int id1 , String name1)
    {
        setId (id1);
        setName (name1);
    }

    public void setId (id1)
    {
        id = id1;
    }

    public void setName (String name1)
    {
        name = name1;
    }
}
```

```

public int getId()
{
    return id;
}
public String getName()
{
    return name;
}
public String toString()
{
    return getId() + " " + getName();
}
public static void main (String args[])
{
    Student s1 = new Student (100, "ram");
    System.out.println(s1);
}
}

```

output:- 100 ram.

- Q) Write a java program define a Cuboid class and data members l, b, h and write the setter and getter methods and calculate total surface area and volume and write three argument constructor for the l, b, h.

sol: package pl;

public class Cuboid

{
 public int l, b, h;

 public static int res1, res2;

 public void setL(int l1)

 {
 l = l1;

 }

 public void setB(int b1)

 {
 b = b1;

 }

 public void setH(int h1)

 {
 h = h1;

 }

 Cuboid(int l1, int b1, int h1)

 {
 setL(l1);

 setB(b1);

 setH(h1);

 }

 public int totalSurface()

 {
 res1 = 2 * (getL() * getB() + getB() * getH()
 + getH() * getL());

 return res1;

 }

```

public int volume()
{
    res2 = (getL() * getB() * getH());
    return res2;
}

public int getL()
{
    return l;
}

public int getB()
{
    return b;
}

public int getH()
{
    return h;
}

public static void main(String args[]) {
    Cuboid obj = new Cuboid(2,3,4);
    obj.totalSurface();
    obj.volume();
    System.out.println(res1 + " " + res2);
}
}

```

output:- 52 24.

validation :-

- 1) The first validation is the id should contain 5 digits. if we have 5 digits then we assign it to data member or else assign the id value to 0.
 - 2) The name should contain only alphabets, it could not contain any other digits or special symbols. if it is so we assign the name to data member or else assign the NULL value.
- Q) write a java program where class name is student and data members are id, name. Then apply the validation over the id so that id will contain 5 digits.

sol:-

```
package p1;

public class valid
{
    public int id;

    valid (int id1)
    {
        if (validateid (id1))
        {
            this.id = id1;
        }
        else
        {
            this.id = 0;
        }
    }
}
```

```
public boolean validateid (int id1)
```

```
{
```

```
    int n = id1;
```

```
    int temp, count = 0;
```

```
    while (n > 0)
```

```
    {
```

```
        temp = n % 10;
```

```
        count++;
```

```
        n = n / 10;
```

```
    }
```

```
    return (count == 5 ? true : false);
```

```
}
```

```
public String toString()
```

```
{
```

```
    return id + "";
```

```
}
```

```
public static void main (String[] args)
```

```
{
```

```
    valid obj = new valid (23456);
```

```
    System.out.println (obj);
```

```
}
```

```
}
```

Output :- 23456.

Q) write the java program to print name of validation that contains all alphabets or not.

```
sol: package p1;
public class valid
{
    public String name;
    valid (String name1)
    {
        if (validatname (name1))
        {
            this.name = name1;
        }
        else
        {
            this.name = "null";
        }
    }
    public boolean validatname (String name1)
    {
        char k;
        for (int i=0; i<name1.length(); i++)
        {
            k = name1.charAt(i);
            if (!((k >= 'a' && k <= 'z') || (k >= 'A' && k <= 'Z')))
            {
                return false;
            }
        }
        return true;
    }
}
```

```

public String toString()
{
    return name;
}

public static void main (String[] args)
{
    valid obj = new valid ("ram");
    System.out.println (obj);
}
}

```

output:- ram.

Q) write a java program of class Cuboid and data members l,b,h and write a constructor and write a function called volume and apply validation over l,b,h such that no one is negative.

Sol:-

```

package p1;

public class Cuboid
{
    public int l,b,h,vol;

    Cuboid (int l1 , b1 , int h1)
    {
        if (validate (l1) && validate (b1) && validate h)
        {
            this.l = l1;
            this.b = b1;
            this.h = h1;
        }
    }
}

```

```

else
{
    l = b = h = 0;
}
volume(l, b, h);
}
public boolean validate (int k)
{
    if (k > 0)
    {
        return true;
    }
    else
    {
        return false;
    }
}
public String toString()
{
    return vol + " " + " ";
}
public static void main (String[] args)
{
    Cuboid obj1 = new Cuboid (2, 3, 4);
    System.out.println (obj1);
    Cuboid obj2 = new Cuboid (-2, 3, 4);
    System.out.println (obj2);
}
}

```

output :- 24

Access specifier and Access Modifier:-

It specifies the accessibility and scope of a variable & method & class. we have

4 access modifiers : i) Private

ii) public

iii) Default

iv) Protected

Access specifier	with class (C ₁)	with in package C ₂	with with package & class (C ₃)	Another packages C ₄
Private	Yes	No	No	No
Default	yes	yes	NO	NO
Protected	yes	yes	yes	NO
Public	yes	yes	yes	yes.

* Private members are accessed with in the class only.

* Public members are accessed any where.

* Default can be accessed with in same package

* Protected can be accessed with in the class, same package, diff package with sub class.

Q) write a java program which demonstrate the access modifier.

Sol: Examples :-

Case-1 :- (with in class).

```

package pl;
public class access
{
    private int a;
    int b;
    protected int c;
    public int d;
    public static void main(String[] args)
    {
        access obj = new access();
        System.out.println(obj.a);
        System.out.println(obj.b);
        System.out.println(obj.c);
        System.out.println(obj.d);
    }
}

```

case-2 :- (with in package)

```

package pl;
public class B
{
    public static void main(String[] args)
    {
        access obj = new access();
        System.out.println(obj.b);
        System.out.println(obj.c);
        System.out.println(obj.d);
        // System.out.println(obj.a); → can't
    }
}

```

access private

Case 4:- (Another package)

```
package p2;
```

```
import p1.access;
```

```
public class C
```

```
{ public static void main (String[] args)
```

```
{ access ob2 = new abaccess();
```

```
System.out.println (ob2.d);
```

```
System.out.println (ob2.a);
```

```
System.out.println (ob2.b);
```

```
System.out.println (ob2.c);
```

} → can't access
default,
private,
protected.

Inheritance :-

It is a mechanism to acquire the properties of one class to another class. The purpose of inheritance is code reuseability.

Reuseability:- write once and reuse anywhere

Inheritance are different types

1) Single

2) Multilevel

3) Hierarchy

4) Multiple

5) hybrid.

Inheritance use extends.

Eg: A $\longrightarrow$ Super / parent
 $\downarrow$ extends
 B $\longrightarrow$ sub / child

Q) write a java program which demonstrate the inheritance on single.

Sol:-

Single inheritance :

If any class acquires the properties from only one class then we can say that single inheritance.

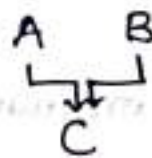
Single



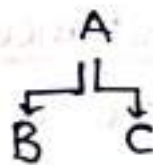
Multilevel



Multiple



heritance



Eg:-

```

package p1;
public class A
{
    public int a = 10;
    public int b = 20;
    public void methodA()
    {
        System.out.println ("Method A");
    }
}
  
```

```
public class B extends A
```

```
{
```

```
    public int c = 30;
```

```
    public void method B()
```

```
    {
```

```
        System.out.println("Method B");
```

```
    }
```

```
    public static void main (String[] args)
```

```
    {
```

```
        B obj = new B();
```

```
        System.out.println(obj.a);
```

```
        System.out.println(obj.b);
```

```
        System.out.println(obj.c);
```

```
        obj.Method A();
```

```
        obj.Method B();
```

```
    }
```

```
}
```

output:

30

20

10

Method A

Method B

Multilevel inheritance :-

```
class A
{
}
```

```
class B extends A
{
}
```

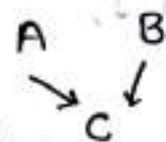
```
class C extends B
{
}
```

```
class D extends C
{
}
```



Multiple :-

```
class C extends A, B
{
}
```



→ (X) (not possible)

Java doesnot Support multiple inheritance in class level.

Java Supports multiple inheritance in interface level.

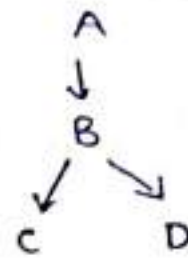
Eg:-

```
class A
{
}
```

```
class B extends A
{
}
```

```
class C extends B
{
}
```

```
class D extends B
{
}
```



Constructor working in inheritance :-

It will execute the first parent class constructor then only it will execute the child class constructor.

Zero argument constructor of parent class is called automatically.

Q) write a java program which demonstrate the constructor execution in inheritance.

Sol:- package p1;

```
class A()
```

```
{
    A()
```

```
{
    System.out.println("con A");
}
```

```

    public void fun1 ( )
    {
        System.out.println ("fun 1 -");
    }
}

```

```

class B extends A

```

```

{
    B ( )
    {
        System.out.println ("con B ");
    }
}

```

```

public class Test

```

```

{
    public static void main (String[] args) {
        B obj = new B ();
        obj.fun1 ();
    }
}

```

output : con A
 con B
 fun1

Constructors will not come down in inheritance as well as static methods also.

Super Keyword :-

super keyword is used to call the parent class constructor and also used to call the normal functions also of parent class.

super keyword should be the first line.

Eg :-

```
package pl ;
class A
{
    int id;
    A()
    {
        id = 0 ;
    }
    A(int id1)
    {
        id = id1 ;
    }
}
class B extends A
{
    String name;
    B()
    {
        name = " ";
    }
    B(int id1, String name1)
    {
        super(id1);
        name = name1 ;
    }
}
```

```

    }
    public String toString()
    {
        return id + " " + name;
    }
    public class Test
    {
        public static void main (String[] args)
        {
            B ob = new B (1234, "ram");
            System.out.println (ob);
            B ob1 = new B ();
            System.out.println (ob1);
        }
    }

```

output: 1234 ram
0 -

for calling functions:

```

    public void fun2()
    {
        Super.fun1();
        System.out.println ("Method is fun2");
    }

```

Q) write a java program of class student details of data members name, id, phno, email and create another child class m1, m2, m3, and calculate the percentage.

Sol: package p1;

class Studentdetails

```
{
    int id, res;
    String name, email;
    long phno;
    Studentdetails()
    {
        id = 0;
        name = "";
        email = "";
        phno = 0;
    }
    Studentdetails (int id1, String name1, long phno1,
                    String email1)
    {
        name = name1;
        id = id1;
        phno = phno1;
        email = email1;
    }
}
```

class Studentmarks extends Studentdetails

```
{
    int m1, m2, m3;
    Studentmarks()
    {
        m1 = m2 = m3 = 0;
    }
}
```

```

Studentmarks (int id1, string name1, long phno1,
{
    String email1, int a, int b, int c)
    Super (id1, name1, phno1, email1);
    m1 = a;
    m2 = b;
    m3 = c;
}

```

```

    public void percentage()
    {
        res = (m1+m2+m3)/3;
    }
}

```

```

public class Student
{
    public static void main (String[] args)
    {

```

```

        Studentmarks obj = new Studentmarks (
            31333, "Abc", 956, "mail",
            94, 95, 96);

```

```

        obj . percentage ();

```

```

        System.out.println (obj);
    }
}

```

write toString after percentage() to return value.

Function overriding:

In child class we rewrite (or) remodify the parent class function then we called it as function overriding (the name should be same and no. of arguments should be same).

Eg:

```
class A
{
    public void fun()
    {
        System.out.println("fun A");
    }
}
class B extends A
{
    public void fun()
    {
        System.out.println("fun B");
    }
}
```

Here fun function is overridden.

Q) write a java program which demonstrate the function overriding (Single inheritance and function overriding).

Sol:

```
package P1;
class A
{
    public void fun()
    {
        System.out.println("Fun of class A");
    }
}
```

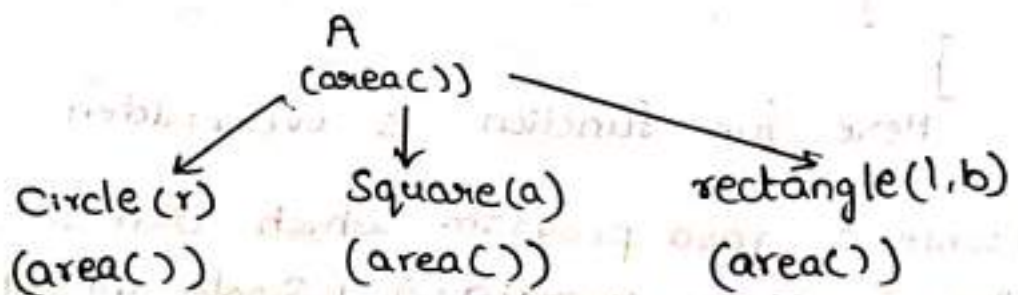
```

class B extends A
{
    public void fun ()
    {
        System.out.println("Fun of class B");
    }
}

public class test
{
    public static void main (String[] args)
    {
        B obj = new B ();
        obj.fun ();
    }
}

```

Q) write a java program according to below like hierarchy and function overriding.



sol:-

```

package p1;

class A
{
    A ()
    {
    }
}

```

```

    public void area()
    {
    }
}
class circle extends A
{
    int r;
    circle()
    {
        r = 3;
    }
    public void area()
    {
        System.out.println(3.14 * r * r);
    }
}

```

```

class square extends A
{
    int a;
    square()
    {
        a = 2;
    }
    public void area()
    {
        System.out.println(a * a);
    }
}

```

```

class rectangle extends A
{
    int l, b;
}

```

```
rectangle()
```

```
{
```

```
    l = 2 ;
```

```
    b = 4 ;
```

```
}
```

```
public void area()
```

```
{
```

```
    System.out.println (l*b);
```

```
}
```

```
}
```

```
public class Overriding
```

```
{
```

```
    public static void main (String[] args)
```

```
{
```

```
        circle ob1 = new circle();
```

```
        ob1.area();
```

```
        Square ob2 = new Square();
```

```
        ob2.area();
```

```
        rectangle ob3 = new rectangle();
```

```
        ob3.area();
```

```
}
```

```
}
```

Output :- 28.259999999998

4

8

2) what is mutators in java?

sd: Setters and getters are called as mutators and accessors.

Q) What are the rules for defining a constructor?

Sol:- 1) The name of the constructor is same as class name.

2) No return type (even void). It is called automatically whenever object is created.

3) You should not write static, final, synchronized for constructor.

Q) Is it possible to call a subclass constructor from super class constructor.

Sol:-

Dynamic method dispatch / run time polymorphism :-

```
class A
{
    public void Fun()
    {
        System.out.println("Fun 1");
    }
}

public void main(String[] args)
{
    A obj = new A();
    obj.Fun();
}
```

In above code `obj.Fun()` calling statement binds with the function definition that will be decide at Compile time.

In dynamic dispatch calling a function binds to the function definition at runtime.

Dynamic dispatch should have :-

- 1) Inheritance
- 2) Function overriding
- 3) Parent can hold the child class object.

A
↓
B

```
A a1;
B b1 = new B();
a1 = b1;
```

From the above a parent class can hold the child class. but child can't hold

* $b_1 = a_1$ is not possible.

Q) write a java program which demonstrate dynamic memory dispatch.

Sol:-

```
package p1;  
class A  
{  
    public void Fun()  
    {  
        System.out.println("Function A");  
    }  
}  
class B extends A  
{  
    public void Fun()  
    {  
        System.out.println("Function B");  
    }  
}  
public class Dispatch  
{  
    public static void main (String[] args)  
    {  
        A a1;  
        B b1 = new B();  
        a1 = b1;  
        a1.Fun();  
    }  
}
```

output:- Function B

At runtime the statement `a1.fun()` calls besides to call the child class `Fun` function it will not call based on the type of the object (class `A`).

ArrayList :-

1) `ArrayList` is a class which is defined inside the `java.util` package.

2) `ArrayList` functionality is same as normal array.

3) `ArrayList` is a dynamic nature, it automatically shrink and expand of its size.

4) `ArrayList` will support the generic.

5) generics means which support different kinds of datatypes like `int`, `float`, `double`, another class.

6) `ArrayList` have some important methods

1) `add(ele)` — add an element to list

2) `remove(i)` — we can remove specified element at index `i`.

3) `set(i,ele)` — we modified the value of index with `ele`.

4) `size()` — it specifies size of the array.

5) `clear()` — it clear all the elements.

6) `get(index)` — it will give the element at specified index.

`ArrayList li = new ArrayList();` // old fashion

ArrayList Supports wrapper class not primitive datatypes.

ArrayList will store the similar datatype.

Syntax :

ArrayList <integer> ob = new ArrayList <Integer>

Q) write a java program which reads 5 elements and print it.

Sol:- package pl;

import java.util.*;

public class ArrayList

{

public static void main (String[] args)

{

ArrayList <Integer> li = new ArrayList <Integer>

Scanner sc = new Scanner (System.in);

for (int i=0; i<5; i++)

{

li.add (sc.nextInt());

}

li.set(4, 100);

for (int i=0; i<li.size(); i++)

{

System.out.println (li.get(i));

}

```

for(int k: li)
{
    System.out.println(k);
}
}

```

Q) write a java program which reads 5 names and prints all 2 second letter of each word in reverse order. using ArrayList

Sol:

```

package pl;
import java.util.Scanner;
import java.util.ArrayList;
public class ReverseArrayList
{
    public static void main(String[] args)
    {
        ArrayList<String> li = new ArrayList<String>();
        Scanner sc = new Scanner(System.in);
        for(int i=0; i<5; i++)
        {
            li.add(sc.next());
        }
        for(char i=4; i>=0; i--)
        {
            System.out.println(li.get(i).charAt(1));
        }
    }
}

```

Array list which contains the objects:-

Q) write a java program where we create a student class with data members id, name and create a arrayList where we store the objects 3, and print the details.

Sol:-

```
package pl;  
import java.util.*;  
class student  
{  
    int id;  
    String name;  
    student()  
    {  
        id = 0;  
        name = "Null";  
    }  
    student (int id1, String name1)  
    {  
        id = id1;  
        name = name1;  
    }  
    public String toString()  
    {  
        return id + " " + name;  
    }  
}  
public class Test  
{  
    public static void main (String[] args)  
    {  
        ArrayList < student > li = new ArrayList < student >
```

```

Student s1 = new Student(123, "jai");
Student s2 = new Student(124, "Sree");
Student s3 = new Student(125, "ram");
li.add(s1);
li.add(s2);
li.add(s3);
for(int i = 0; i < li.size(); i++)
{
    System.out.println(li.get(i));
}
}
}

```

Q) Write a java program which create a class called book id, book name, book price, book author and no. of pages and book publisher. Under this class these are the data members.

menu (only 5 books)

1. add books
2. print all books
3. print a specific book based on id (search)
4. sort the books using id (bubble sort)
5. find the max price book and print its details
6. Delete a specific book.

Sol:

```
package pl;  
import java.util.*;  
class Book  
{  
    public int id, price, pages;  
    public String name, author;  
    Book()  
    {  
    }  
    Book(int id, String name, int price, String author, int  
    {  
        this.id = id;  
        this.name = name;  
        this.price = price;  
        this.author = author;  
        this.pages = pages;  
    }  
    public String toString()  
    {  
        return id + " " + name + " " + price + " " + author + " " + pages;  
    }  
}  
public class Test  
{  
    ArrayList<Book> li = new ArrayList<Book>();  
    Scanner sc = new Scanner(System.in);  
    public void add()  
    {
```

```
System.out.println("Enter id of book : ");
int id = sc.nextInt();
System.out.println("Enter name of book : ");
String name = sc.next();
System.out.println("Enter price of book : ");
String price = sc.nextInt();
System.out.println("Enter author of book : ");
String author = sc.next();
System.out.println("Enter no. of pages : ");
int pages = sc.nextInt();
Book b1 = new Book(id, name, price, author, pages);
li.add(b1);
```

```
}
public void print()
{
    for(int i=0; i<li.size(); i++)
    {
        System.out.println(li.get(i));
    }
}
public void search()
{
    System.out.println("Enter id to search : ");
    int id1 = sc.nextInt();
    int count = 0;
    for(int i=0; i<li.size(); i++)
    {
```

```

if (id1 == li.get(i).id)
{
    count++;
    System.out.println(li.get(i));
}
}
if (count >= 1)
{
    System.out.println("Book found");
}
else
{
    System.out.println("Book not found");
}
}

public void sort()
{
    for (int i = 0; i < li.size(); i++)
    {
        for (int j = 0; j < li.size() - i - 1; j++)
        {
            if (li.get(j).id > li.get(j+1).id)
            {
                Book temp = new Book();
                temp = li.get(j);
                li.set(j, li.get(j+1));
                li.set(j+1, temp);
            }
        }
    }
}

```

```
public void maxprice()
```

```
{ int i, y=0;
```

```
int x=li.get(0).price;
```

```
for(i=0; i<li.size(); i++)
```

```
{ if (li.get(i).price > x)
```

```
{ x=li.get(i).price;
```

```
y=i;
```

```
}
```

```
}
```

```
System.out.println(li.get(y));
```

```
}
```

```
public void delete()
```

```
{
```

```
System.out.println("Enter id to delete : ");
```

```
int id1=SC.nextInt();
```

```
for(int i=0; i<li.size(); i++)
```

```
{ if (id1 == li.get(i).id)
```

```
{
```

```
li.remove(i);
```

```
}
```

```
}
```

```
}
```

```
public static void main(String[] args)
```

```
{
```

```
Scanner sc= new Scanner(System.in);
```

```
Test t1= new Test();
```

```
int ch = 0;
```

```
while (true)
```

```
{
```

```
System.out.println("1. Add\n 2. print\n 3. Search\n 4. Sort\n 5. Maximum\n   priced book\n 6. Delete a\n   book\n 7. Exit");
```

```
System.out.println("Enter your choice: ");
```

```
ch = sc.nextInt();
```

```
switch (ch)
```

```
{
```

```
case 1 : t1.add();
```

```
break;
```

```
case 2 : t1.print();
```

```
break;
```

```
case 3 : t1.search();
```

```
break;
```

```
case 4 : t1.sort();
```

```
break;
```

```
case 5 : t1.maxprice();
```

```
break;
```

```
case 6 : t1.delete();
```

```
break;
```

```
case 7 : System.exit(0);
```

```
break;
```

```
default : System.out.println("Invalid choice");
```

```
}  
}  
}
```

- Q) Design a class named triangle that extends geometric shape. The class contains
- i) 3 double data fields likes side1, side 2 and side 3 as default value 1.0
 - ii) A no argument Constructor creates default
 - iii) A construct creates that triangle with specified side1, side 2, side 3.
 - iv) The accessor methods for all three data.
 - v) A method named getArea() that returns the area of this triangle
 - vi) A method named getPerimeter() that returns the perimeter of triangle.
 - vii) A method named toString() that returns a string of triangle.
 - viii) write a test program that scans 3 sides of triangle, a color, a boolean value to indicate whether triangle is filled or not.
- The program should create a Triangle object with these sides and set the colour and filled properties using input.
- The program should display the area, perimeter, colour, and true or false to indicate whether it is filled or not.
- Draw the class diagram also for the above problems.

```

package pl;
import java.util.*;
class GeometricShape
{
    double side1, side2, side3, area, perimeter;
    boolean isfilled;
    String colour;
    GeometricShape()
    {
        side1 = side2 = side3 = 1.0;
    }
}
public class Triangle extends GeometricShape
{
    Scanner sc = new Scanner (System.in);
    Triangle(double side1, double side2, double side3,
    {
        String colour, boolean isfilled)
        {
            setSide1(side1);
            setSide2(side2);
            setSide3(side3);
            if (validateColour(colour))
            {
                this.colour = colour;
            }
            else
            {
                this.colour = "null";
            }
        }
    }
}

```

```
if (isfilled == true)
{
    this.isfilled = true;
}
else
{
    this.isfilled = false;
}
```

```
}
public void setSide1 (double side1)
```

```
{
    this.side1 = side1;
}
```

```
public void setSide2 (double side2)
```

```
{
    this.side2 = side2;
}
```

```
public void setSide3 (double side3)
```

```
{
    this.side3 = side3;
}
```

```
public boolean validateColour (String colour)
```

```
{
    if (colour != null)
    {
        return true;
    }
```

```
else
```

```
{
    return false;
}
```

```
}
public double getArea()
```

```
{
```

```
double s = (side1 + side2 + side3) / 2;  
area = Math.sqrt(s * (s - side1) * (s - side2) * (s - side3));  
return area;  
}
```

```
public double getPerimeter()  
{  
    perimeter = side1 + side2 + side3;  
    return perimeter;  
}
```

```
public String toString()  
{  
    return area + " " + perimeter + " " + colour + " " +  
        isfilled;  
}
```

```
public static void main (String[] args)  
{
```

```
    Scanner sc = new Scanner (System.in);  
    double side1 = sc.nextDouble();  
    double side2 = sc.nextDouble();  
    double side3 = sc.nextDouble();  
    System.out.println("Enter any colour : ");  
    String colour = sc.next();  
    boolean isfilled = sc.nextBoolean();
```

```
    Triangle obj = new Triangle (side1, side2,  
        side3, colour, isfilled);
```

```
    obj.getArea();  
    obj.getPerimeter();  
    System.out.println(obj);
```

```
}
```

Aggregation and Composition :-

Inheritance is - a relationship

Eg: Employee extends programmer
programmer is a employee

One class is related to another class in two ways.

1) Inheritance (is-a relationship)

2) Association (has-a relationship)

Aggregation :-

In aggregation you have two class A, B

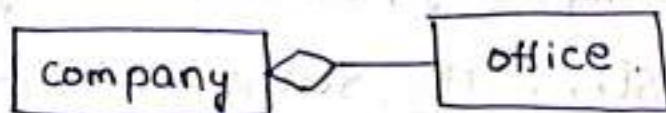
→ In B class contains the object of A class as a data member

→ Aggregation is a lose coupling for link.

Eg:- company has a office

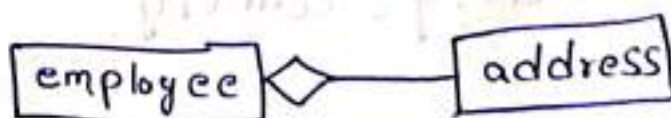
```
Company                office
{
  office ol;
}
```

representation :-



if the parent is deleted then child still exists then its a aggregation.

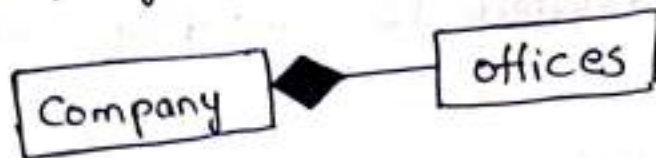
Eg:- Employee has address



Composition :

Composition contains two classes A, B where A has B as data member and coupling is strong coupling.

Eg:- Company has offices



Note :-

In programatically, we don't have any difference between Aggregation and Composition.

Q) write a java program which creates class called Address with data members city, state, country and write another class called employee and data members id, name, address for 2 employee and print the data.

Sol:- package pl;

public class Address

{ public String city, state, country;

Address (String city, String state, String
Country)

{ this.city = city;

this.state = state;

this.country = country;

}

}

```
package pl;
```

```
public class Employee
```

```
{ int id;
```

```
  String name;
```

```
  Address add;
```

```
  Employee (int id, String name, Address add) {
```

```
    this.id = id;
```

```
    this.name = name;
```

```
    this.add = add;
```

```
  } public void display ()
```

```
{ System.out.println(id + " " + name + " " + add.city +  
  " " + add.state + " " + add.country);
```

```
  } public static void main (String[] args)
```

```
{ Address a1 = new Address ("Kkd", "ap", "India");
```

```
  Address a2 = new Address ("rcpm", "ap", "india");
```

```
  Employee e1 = new Employee (1, "abc", a1);
```

```
  Employee e2 = new Employee (2, "def", a2);
```

```
  e1.display();
```

```
  e2.display();
```

Another example for aggregation:-

```
package pl;
```

```
class Operation
```

```
{
```

```
  public int Square (int n)
```

```
{ return n*n;
```

```
  }
```

class Circle

```
{  
    operation op;  
    double pi = 3.14;  
    double area (int radius)  
    {  
        op = new Operation();  
        int r = op.square(radius);  
        return pi * r;  
    }  
    public static void main (String[] args)  
    {  
        Circle c = new Circle();  
        double result = c.area(5);  
        System.out.println(result);  
    }  
}
```

output : 78.5

- Q) write a java program where we define classes called Circle, Rectangle, Square and then create the calculator class where we define 3 datamembers and main method. Inside the main method calculate the area.

sol:- package P1;

class Circle

{
int r;

double pi = 3.14;

Circle (int r)

{

this.r = r;

}

public void area()

{

System.out.println(pi * r * r);

}

}

class Rectangle

{

int l, b;

Rectangle (int l, int b)

{

this.l = l;

this.b = b;

}

public void area()

{

System.out.println(l * b);

}

}

class Square

{

int a;

Square (int a)

{

this.a = a;

}

```

    public void area()
    {
        System.out.println(a*a);
    }
}

public class Calculator
{
    Circle c;
    Square s;
    Rectangle r;
    public void fun()
    {
        c = new Circle(10);
        s = new Square(10);
        r = new Rectangle(5,10);
        c.area();
        s.area();
        r.area();
    }

    public static void main(String[] args)
    {
        Calculator c1 = new Calculator();
        c1.fun();
    }
}

```

output :-
 314.0
 100
 50.

2) write a java program which describes a menu driven program on Aggregation as well as array of objects for the student class and menu is

- i) addstudent
- ii) print all students
- iii) Exit.

solt:

```
package p1;  
import java.util.*;  
class student3  
{  
    int id;  
    String name;  
    student3(int id, String name)  
    {  
        this.id = id;  
        this.name = name;  
    }  
    public String toString()  
    {  
        return id + " " + name;  
    }  
}
```

```
public class Menu - Aggregation  
{  
    int c=0;  
    Student3 ob[] = new Student3[5];  
    public void add()  
    {  
        Scanner sc = new Scanner(System.in);
```

```

int id = sc.nextInt();
String name = sc.next();
ob[c] = new Student3(id, name);
c++;
System.out.println("Inserted");
}

```

```

public void print()
{
    for (int i = 0; i < c; i++)
    {
        System.out.println(ob[i]);
    }
}

```

```

public static void main (String[] args)
{

```

```

    Menu-Aggregation obj = new Menu-Aggregation();
    Scanner sc = new Scanner(System.in);

```

```

    int ch = 0;

```

```

    while (true)
    {

```

```

        System.out.println("1. Add a Student");

```

```

        System.out.println("2. Print all \n 3. Exit");
        System.out.println("Enter choice: ");

```

```

        ch = sc.nextInt();

```

```

        switch (ch)
        {

```

```

            case 1:

```

```
int id = sc.nextInt();  
String name = sc.next();  
ob[c] = new Student3(id, name);  
c++;  
System.out.println("Inserted");  
}
```

```
public void print()  
{  
    for (int i = 0; i < c; i++)  
    {  
        System.out.println(ob[i]);  
    }  
}
```

```
public static void main (String[] args)
```

```
{  
    Menu-Aggregation obj = new Menu-Aggregation();  
    Scanner sc = new Scanner(System.in);  
    int ch = 0;  
    while (true)  
    {  
        System.out.println("1. Add a Student  
                             2. Print all \n 3. Exit");  
        System.out.println("Enter choice : ");  
        ch = sc.nextInt();  
        switch (ch)  
        {  
            case 1:  
                obj.addStudent();  
                break;  
            case 2:  
                obj.print();  
                break;  
            case 3:  
                return;  
            default:  
                System.out.println("Invalid choice");  
        }  
    }  
}
```

```
case 1 : obj.add();  
        break;
```

```
case 2 : obj.print();  
        break;
```

```
case 3 : System.exit(0);  
        break;
```

```
default : System.out.println("Invalid choice")
```

```
}  
}  
}  
}
```

```
case 1 : obj.add();  
        break;
```

```
case 2 : obj.print();  
        break;
```

```
case 3 : System.exit(0);  
        break;
```

```
default : System.out.println("Invalid choice");
```

```
}  
}  
}
```

* Singleton class :-

1) It will create only one object that object is shared among the different request.

2) Singleton point always ensure creation of only one object.

3) Request means it always says that provide the object.

4) Rules for the singleton class

i) Create a data member such that provide it should be private static single object

ii) Create all private constructors

iii) Create a method called getInstance public static.

iv) you will request the class by using
get instance() method.

v) Actually we have 2 types of Singleton
class

① compile time

② Run time.

Q write a java program which demonstrates
the early singleton class.

Sol:-

```
package p1;
```

```
public class singlec  
{
```

```
    private static singlec ob = new singlec();
```

```
    private String str;
```

```
    private singlec ()
```

```
    {  
        str = "this is compile time Singleton";
```

```
    }  
    public void display()
```

```
    {  
        System.out.println(str);
```

```
    }  
    public static singlec get Instance()
```

```
    {
```

```
        return ob;
```

```
    }
```

```

public static Singlec void main (String args[])
{
    Singlec obj = get Instance ();
    obj.display ();
    Singlec ob = get Instance ();
    ob.display ();
}
}

```

Compile time :-

Before the executing the main method ob object is created that is called Compile time.

After the main method execution if ob objects created then we call it as Run time.

Q) write a java program which demonstrate Run time singleton class.

sol:-

```

public class Singlec
{
    private static Singlec ob;
    private String str;
    private Singlec ()
    {
        str = "this is runtime Singleton";
    }
}

```

```

public void display()
{
    System.out.println(str);
}

public static Singlec getInstance()
{
    if (ob == NULL)
    {
        ob = new Singlec();
        return ob;
    }
    return ob;
}

```

output :



Abstract class:-

A abstract class is same as class which contains abstract methods as well as normal methods (concrete methods).

Abstract method:-

A method which doesnot have a body is called abstract method.

Concrete method:-

A method which have a body is called concrete method.

Eg:- void add(); → abstract method.

This should written as

abstract void add();

void sub()

{
 ==
}
} → Concrete method.

Abstraction:-

Hiding the implementation details and showing only essential details is called as abstraction.

Rules for the abstract class:-

1) we cannot create object for the abstract class.

2) Atleast one method is abstract method then the class has to be declared abstract class.

3) In abstract class, the abstraction range will be 0 to 100%.

4) A class which extends abstract class must implement all the abstract methods, if not then the class will become abstract class.

5) Abstract class can have a constructor as well as static method.

Eg:-

abstract class A

{
 abstract void M1();
 void M2();
}

 //
 //
 //
}

class Test extends A

{
 void M1()
 {
 //
 //
 //
 }

 }
}

};

5) write a java program which demonstrate the abstract class and abstract method.

```
package pl;  
  
abstract class A  
{  
    abstract public void fun1();  
    abstract public void fun2();  
    public void fun3()  
    {  
        System.out.println("fun 3");  
    }  
}
```

```
public class test extends A  
{  
    public void fun1()  
    {  
        System.out.println("fun1");  
    }  
    public void fun2()  
    {  
        System.out.println("fun 2");  
    }  
}
```

```
public static void main (String[] args)  
{  
    test ob = new test();  
    ob.fun1();  
    ob.fun2();  
    ob.fun3();  
}
```

```

A a ;
a = new test ();
a.fun1 ();
a.fun2 ();
a.fun3 ();
}
}

```

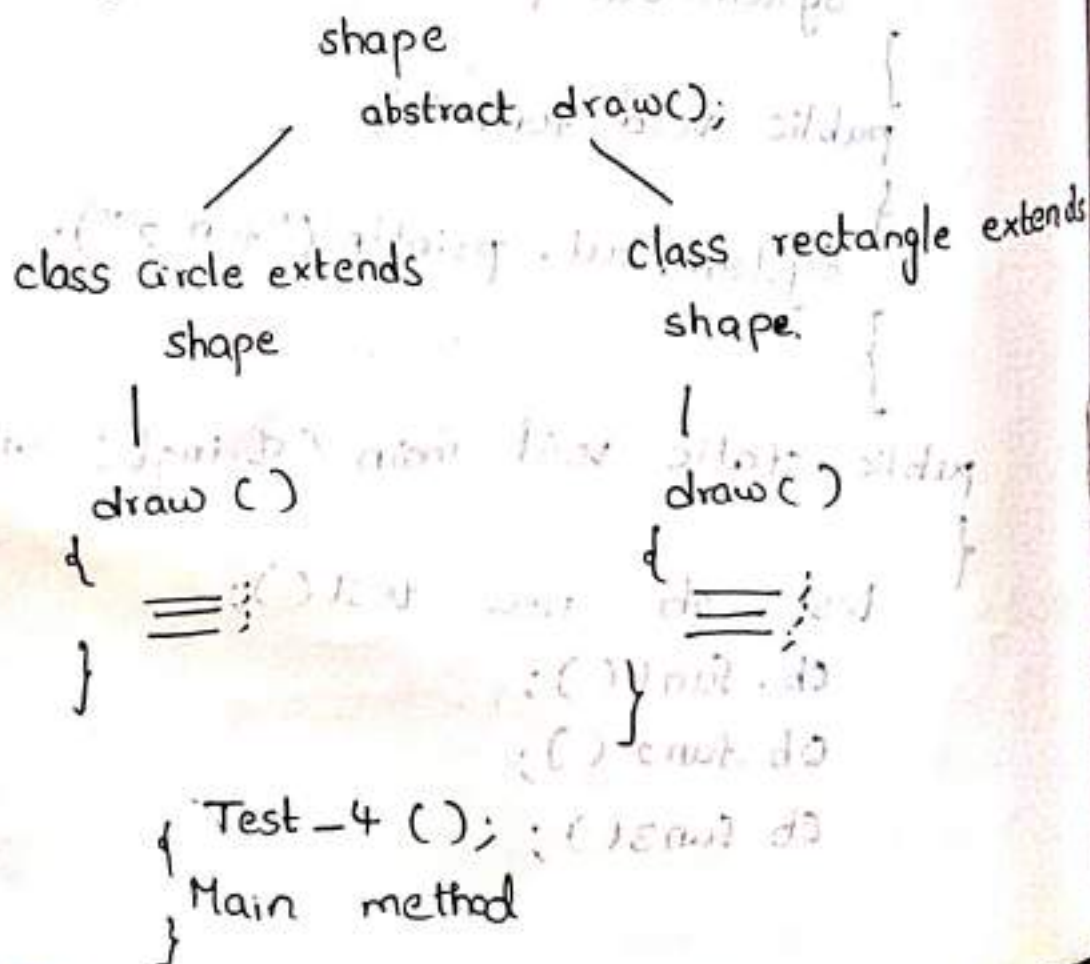
output :-

```

fun 1
fun 2
fun 3
fun 1
fun 2
fun 3.

```

Q) write a java program to implement the below diagram:



```
package pl;
```

```
abstract class shape
```

```
{
```

```
    abstract public void draw();
```

```
}
```

```
public class Circle extends shape
```

```
{
```

```
    public void draw()
```

```
    {  
        System.out.println("Circle");  
    }
```

```
}
```

```
public class rect extends shape
```

```
{
```

```
    public void draw()
```

```
    {  
        System.out.println("Rectangle");  
    }
```

```
}
```

```
public class test4
```

```
{
```

```
    public void static void main(String[] args)
```

```
    {
```

```
        shape s;
```

```
        s = new Circle();
```

```
        s.draw();
```

```
        s = new rect();
```

```
        s.draw();  
    }
```

Final Keyword:-

Final keyword can be used in 3 ways

- i) with class
- ii) constant
- iii) method.

with class:-

A class which is declared final, it cannot be extended.

⇒ A class can't be defined as abstract and final. (Both at a time).

Example for final method:-

① Final method cannot be overridden.

Eg:-

```
package pl;  
class M  
{  
    public final void fun1()  
    {  
        System.out.println("fun1");  
    }  
}  
public class test extends M  
{  
    Error ← public void fun1()  
    {  
        System.out.println("Child: fun1");  
    }  
    public static void main (String[] args) {  
    }  
}
```

constant :-

```
public final int pi = 3 ;
```

Interface :-

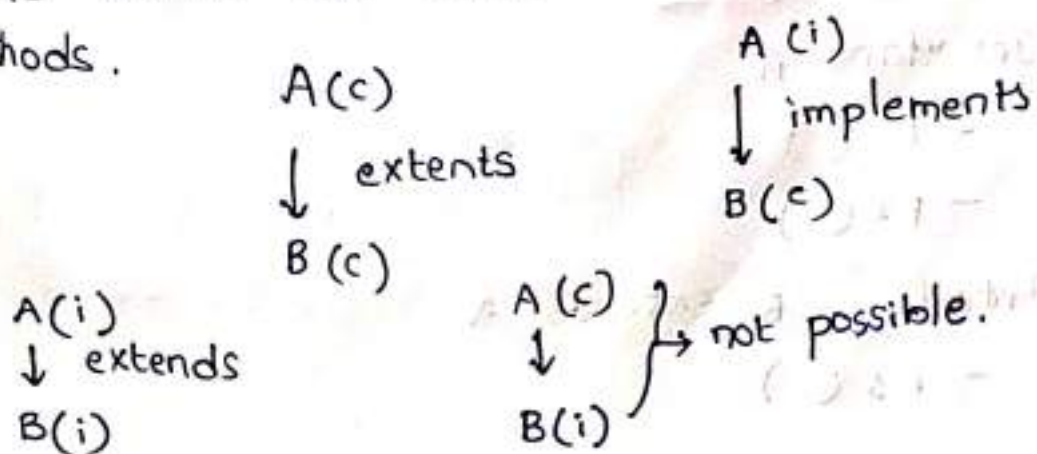
Interface is same as class, with abstraction level is 100%. but in java 8 onwards they introduced a default and static method in interface.

- 1) By default all the variables of interface is public static final variables.
- 2) By default all the methods of interface is public abstract methods.
- 3) Interface, we should not write constructor.

Syntax :-

```
interface B
{
    int a ;
    int b ;
    public void add ( ) ;
}
```

- 3) we cannot create object for interface.
- 4) A class which implement interface must and should write the code for all the abstract methods.



Q) write a java program which implements the addition of 2 numbers using interface

Sol:

```
package pl;  
interface K  
{  
    int a = 2;  
    int b = 3;  
    public void add();  
}  
public class test5 implements K  
{  
    public void add()  
    {  
        System.out.println(a+b);  
    }  
    public static void main (String[] args)  
    {  
        test5 ob = new test5();  
        ob.add();  
    }  
}
```

output :- 5.

Q) Interface A

- F1()

- F2()

Interface B extends A

- F3()

Test extends implements B
main().

package pl;

interface A

{
 public void f1();
 public void f2();
}

interface B extends A

{
 public void f3();
}

public class test implements B

{
 public void f1()

{
 System.out.println("f1");
 }

public void f2()

{
 System.out.println("f2");
 }

public void f3()

{
 System.out.println("f3");
 }

public static void main(String[] args)

{
 test ob = new test();

ob.f1();

ob.f2();
}

ob.f3();

```
}  
}
```

output :- f1

f2

f3.

Note :-

Interface supports multiple inheritance but
class does not support multiple inheritance

Multiple inheritance in interface level:-

Eg:-

package P1;

interface A

{

show();

}

interface B

{

display();

}

public ~~test~~ implement A, B

{

public void show()

{

System.out.println("show");

}

```

public void display()
{
    system.out.println("display");
}

public static void main (String[] args)
{
    test ob = new test();
    ob.show();
    ob.display();
}

```

Write a java program for your previous problem shape and implement using interface

```

shape (I)
- draw()
- calarea()

Circle imple
draw
calarea()

draw
calarea()

```

sol:

```

package pl;

interface shape
{
    public abstract void draw();
    public abstract void calarea();
}

```

class circle implements shape

```
{
    int r=2;
    public void draw()
    {
        System.out.println("Circle");
    }
    public void calarea()
    {
        System.out.println("3.14 * r * r");
    }
}
```

class rect implements shape

```
{
    int a = 3, b = 4;
    public void draw()
    {
        System.out.println("rectangle");
    }
    public void calarea()
    {
        System.out.println(a*b);
    }
}
```

public class test

```
{
    public static void main(String[] args)
    {

```

```

shape s;
s = new circle();
s.draw();
s.calarea();
s = new rect();
s.draw();
s.calarea();
}
}

```

output : Circle
 12.56
 rectangle
 12

Vector and linked list :-

A Vector is a class name.

- i) It is same as ArrayList, except vector is a Synchronized and ArrayList is not Synchronized.
- ii) Vector is a dynamic array, whenever you add element it automatically expands.
- iii) Vector also supports Generic i.e. it will support different data types.
- iv) Different methods in the Vector
 - i) get()
 - ii) add()
 - iii) remove(index)

```

shape s;
s = new circle();
s.draw();
s.calculate();
s = new rect();
s.draw();
s.calculate();
}
}

```

output : Circle
 12.56
 rectangle
 12

Vector and linked list :-

A Vector is a class name.

- i) It is same as ArrayList, except vector is a Synchronized and ArrayList is not Synchronized.
- ii) Vector is a dynamic array, whenever you add element it automatically expands.
- iii) Vector also supports Generic i.e. it will support different data types.
- iv) Different methods in the vector
 - i) get()
 - ii) add()
 - iii) remove(index)

iv) lastElement()

v) removeAll()

vi) set(index, new value)

vii) Size

viii) Capacity()

ix) indexOf(value)

x) clear()

xi) Contains()

xii) add(2, value)

xiii) firstElement()

xiv) lastIndexOf()

Q) write a java program which demonstrate using Vector class.

Sol:-

```
public class VectorDemo
{
    public static void main (String[] args)
    {
        Vector < Integer > obj = new Vector
        < Integer > ();
        obj.add(10);
        obj.add(20);
        obj.add(30);
        obj.add(40);
        obj.add(50);
        System.out.println (obj.capacity());
    }
}
```

```

for (Integer t : obj)
{
    System.out.println(t);
}
System.out.println(obj.IndexOf(10));
System.out.println("After removal");
obj.remove(2);
for (Integer t : obj)
{
    System.out.println(t);
}
obj.set(2, 30);
System.out.println("After updation");
for (Integer tmp : obj)
{
    System.out.println(tmp);
}
System.out.println("First element : ");
System.out.println(obj.firstElement());
System.out.println("Last element : " +
                    obj.lastElement());
for (int i = 0; i < obj.size(); i++)
{
    System.out.println(obj.get(i));
}
}

```

Q) write a java program where we define a Student class and the Student Contains id, name and marks m1, m2, m3 and the phno. You Create a Vector

- i) add all 5 students to the vector
- ii) modify the 2nd student details
- iii) Remove all student
- iv) Print 1st student details and check whether a particular student is there or not. (based on name)
- v) print the all student details in the short form of the for loop.

Sol:-

```
package pl;  
import java.util.Vector;  
public class Student  
{  
    int id;  
    String name;  
    int m1, m2, m3;  
    long phno;  
    Student()  
    {  
        id = 0;  
        name = "null";  
        m1 = m2 = m3 = 0;  
        phno = 0;  
    }  
}
```

```
Student (int id, String name, int m1, int m2, int m3,  
        long phno)
```

```
{  
    this.id = id;  
    this.name = name;  
    this.m1 = m1;  
    this.m2 = m2;  
    this.m3 = m3;  
    this.phno = phno;  
}
```

```
public String toString()  
{  
    return id + " " + name + " " + m1 + " " + m2 + " " +  
           m3 + " " + phno;  
}
```

```
public static void main(String[] args)
```

```
{  
    Vector<Student> obj = new Vector<Student>();
```

```
    Student s1 = new Student(31333, "Phanindra", 96, 94,  
                              92, 9392978076);
```

```
    Student s2 = new Student(31026, "Jaswanth", 99, 97,  
                              95, 8974293102);
```

```
    Student s3 = new Student(30041, "Ganesh", 96, 94,  
                              99, 8332059888);
```

```
    Student s4 = new Student(31576, "Abhiram", 93, 94,  
                              96, 932565239);
```

```
    Student s5 = new Student(31350, "Ramana", 99, 98,  
                              97, 630325614);
```

```
    obj.add(s1);
```

```
    obj.add(s2);
```

```
obj.add(s3);
```

```
obj.add(s4);
```

```
obj.add(s5);
```

```
Student s6 = new Student(30621, "Karthikeya",  
95, 98, 100, 85649812);
```

```
obj.set(1, s6);
```

```
obj.remove(obj.indexOf(s5));
```

```
System.out.println(obj.get(obj.indexOf(s1)));
```

```
int flag = 0;
```

```
for (Student k : obj)
```

```
{  
    if (k.name.equals("Karthikeya"))
```

```
{  
        flag = 1;  
        break;
```

```
}  
if (flag == 1)
```

```
{  
    System.out.println("Yes");
```

```
else
```

```
{  
    System.out.println("No");
```

```
}  
for (Student i : obj)
```

```
{  
    System.out.println(i);
```

```
}  
}
```

GUI (Graphical User Interface):

Graphical user Interface has a packages they are

- i) AWT (Abstract window tool kit)
- ii) Swings

awt - java.awt.*;

swings - java.swing.*;

awt is a old one, it is replaced by swings

awt has a set of components or class. They are

- 1) Label
- 2) Textfield
- 3) Button
- 4) check Box
- 5) radio Button
- 6) choice
- 7) list
- 8) Combo box
- 9) Password

Swings Components are

- | | |
|------------------|---------------|
| 1) JLabel | 6) Jchoice |
| 2) Jtextfield | 7) Jlist |
| 3) JButton | 8) Jcombo box |
| 4) Jcheck Box | 9) JPassword |
| 5) Jradio Button | |

The difference between awt and swings is awt components appearance depends on the operating system. swings components appearance does not depend on operating system.

swings has a more components than awt.

Steps to create the GUI application:-

step-1:- Create a class A extends the Frame.

step-2:- Write code in the A class constructor.

step-3:- Create all required components.

step-4:- Add all components to the frame

[upto step 4 - Decoration]

step-5:- Event Driven Model (Performing the action). $\downarrow$ (available in)
`java.awt.event.*;`

Our A class should implements ActionListener that interface contains only one method i.e actionPerformed.

when you click the button, it automatically makes a call to actionPerformed method.

1) click on button is $\rightarrow$ event

listener $\rightarrow$ ActionListener

2) Mouse clicking (event)
Mouse listener (listener).

3) window closing (event)
window listener (listener). / window adapter

public void actionPerformed (ActionEvent A)

b. add ActionListener (this);

↳ to add action to particular button.

Q) write a java program which creates a calculator like below.

Sol:-

module-info.java

```
module com.remordial
{
    requires java.desktop;
}
```

```
package pl;
```

```
import java.awt.*;
```

```
import java.awt.event.*;
```

```
import javax.swing.*;
```

```
public class cal2 extends JFrame
```

```
implements ActionListener {
```

```
    JButton b;
```

```
    JButton x;
```

```
    JLabel
```

```
    JLabel 12;
```

JLabel l3;

TextField tf1;

TextField tf2;

Calc2 ()

b = new JButton("Add");

r = new JButton("Reset");

l1 = new JLabel("Enter a ");

l2 = new JLabel("Enter b ");

l3 = new JLabel("Result: ");

tf1 = new TextField(10);

tf2 = new TextField(10);

add(l1);

add(tf1);

add(l2);

add(tf2);

add(l3);

add(b);

add(r);

tf1.setText("0");

tf2.setText("0");

setSize(800, 500);

setVisible(true);

setTitle("Calculator");

```
setLayout (new FlowLayout());  
setDefaultCloseOperation (EXIT_ON_CLOSE);
```

```
/* addWindowListener (new WindowAdapter())  
{  
    public void windowClosing (WindowEvent e)  
    {  
        dispose();  
    }  
} */
```

```
b. addActionListener (this);
```

```
b1. addActionListener (this);
```

```
}  
public void actionPerformed (ActionEvent ae)  
{  
    int t1, t2;  
    if (ae.getSource() == b)  
    {  
        t1 = Integer.parseInt (tf1.getText());  
        t2 = Integer.parseInt (tf2.getText());  
        int k = t1 + t2;  
        l3.setText ("Result " + Integer.toString(k));  
    }  
    if (ae.getSource() == b1)  
    {  
        tf1.setText ("0");  
        tf2.setText ("0");  
    }  
}
```

```

13. set Text ("Result : 0");
}
}
public static void main (String[] args)
{
    new cal1 ();
}
}.

```

Linked list :-
 Linked list is a class, it is available in util package.

removeFirst () — delete at beg ()
 removeLast () — delete at end ()
 remove (2) — delete at pos ()
 addFirst (val) — insert at beg ()
 add (2, 10) — insert at pos ()
 addLast (val) — insert at end ()
 getFirst (); — display first node
 getLast (); — display last node
 get (2) — display specific node
 clear () — clear all elements.
 indexOf (ele) — find index of element
 set (2, 34) — update the element.
 size () — count ()

- # Linked list is non synchronized and maintain a insertion order
- # Linked list is used to implement stacks and Queues.

Q) write a java program which implements the linked list in Java?

```

package pl;
import java.util. LinkedList;

public class link1
{
    public static void main (String[] args)
    {
        LinkedList<Integer> list = new
                                LinkedList<Integer>
        list.addFirst(40);
        list.addFirst(30);
        list.addFirst(20);
        list.addFirst(10);
        list.set(list.indexOf(30), 70);
        list.removeLast();
        list.addLast(100);
        System.out.println(list.size());
        for (int k : list)
            System.out.println(k);
    }
}

```

In arrays the insertion and deletion takes more time due to increasing size of one by one.

But in linked list the time complexity is very less.

Comparable and Comparative :-

1) It is an interface, defined in Java.lang package.

2) Comparable have one method i.e

```
public int CompareTo (obj)
```

3) purpose of Comparable is inorder to sort based on any data member, we use Comparable (or) Compariter.

Eg:- actual data :-

S ₁	S ₂	S ₃
100	102	101
ram	Krishna	Kiran
Guntur	Guntur	Vijayawada.

Sorted order based on id

S ₁	S ₃	S ₂
100	101	102
ram	Kiran	Krishna
Guntur	Vijawada	Guntur.

steps to create Comparable Sorting:

- 1) Create a class Student which implements Comparable.
- 2) So we need to write the code for compareTo based on any data member.
- 3) call the method Collection.sort()
- 4) Create an ArrayList and add student objects
- 5) call method Collection.sort(), it automatically calls the compareTo();
- 6) write a java program which creates a student class and data members are id, name, address and create 4 student objects, and add to the ArrayList and sort based on the id, using Comparable interface.

```
Sol: package pl;
import java.util.*;
public class student implements Comparable<student>
{
    int id;
    String name, add;
    student(int id, String name, String add)
    {
        id = id;
        this.name = name;
        this.add = add;
    }
}
```

```
public String toString()
```

```
{  
    return id + " " + name + " " + add;  
}
```

```
public int compareTo(student s)
```

```
{  
    if (id == s.id)
```

```
{  
        return 0;
```

```
}  
    else if (id > s.id)
```

```
{  
        return 1;
```

```
}  
    else
```

```
{  
        return -1;
```

```
}
```

```
public static void main(String[] args)
```

```
{
```

```
    ArrayList<student> list = new ArrayList<student>();
```

```
    list.add(new student(100, "ram", "Guntur"));
```

```
    list.add(new student(103, "krishna", "Guntur"));
```

```
    list.add(new student(102, "Kiran", "vjj"));
```

```
    for (student k : list)
```

```
{  
        System.out.println(k);  
    }
```

```
Collections.sort(list);  
for (student k : list)
```

```
{  
    System.out.println(k);  
}
```

```
}
```

```
}
```

Comparator :-

1) It is an interface defined inside the java.util package.

2) We have only one method i.e. compare method.

i.e. have

→ public int compare (student s1, student s2)

Steps to define Comparator :-

1) Create a class student id, name, age, constructor, toString().

2) As we need to write code for compare (s1, s2) based on any data member.

3) Create an array list and add students objects (s1, s2, s3)

4) call the method Collections.sort (array list, objectname) , it automatically call the compare ().

5) Create a class called sortByid which implements Comparator.

Q) write a java program which implements the comparator program?

Sol: package pl;

import java.util.*;

public class Student1

{

int id, age;

String name;

Student1(int id, String name, int age)

{

this.id = id;

this.name = name;

this.age = age;

}

public String toString()

{

return id + " " + name + " " + age;

}

public static void main(String[] args)

{

ArrayList<Student1> list = new ArrayList<Student1>();

list.add(new Student1(100, "ram", 12));

list.add(new Student1(103, "Krishna", 14));

list.add(new Student1(102, "Kiran", 13));

System.out.println("Before Sorting:");

for (Student1 k : list)

System.out.println(k);

```

Sortbyid ob1 = new Sortbyid();
Collections.sort(list, ob1);
System.out.println("Sorting by id : ");
for (Student k : list)
{
    System.out.println(k);
}
Sortbyname ob2 = new Sortbyname();
Collections.sort(list, ob2);
System.out.println("Sorting by name : ");
for (Student k : list)
{
    System.out.println(k);
}
Sortbyage ob3 = new Sortbyage();
Collections.sort(list, ob3);
System.out.println("Sorting by age : ");
for (Student k : list)
{
    System.out.println(k);
}
}

class Sortbyid implements Comparator<Student>
{

```

```

public int compare (Student1 s1, Student1 s2)
{
    if (s1.id == s2.id)
    {
        return 0;
    }
    else if (s1.id > s2.id)
    {
        return 1;
    }
    else
    {
        return -1;
    }
}

```

```

class Sortbyname implements Comparator<Student1>
{
    public int compare (Student1 s1, Student1 s2)
    {
        if (s1.name.compareTo (s2.name) == 0)
        {
            return 0;
        }
        else if (s1.name.compareTo (s2.name) == 1)
        {
            return -1;
        }
        else
        {
            return 1;
        }
    }
}

```

```

class sortbyage implements Comparator<Student>
{
    public int Compare (Student s1, Student s2)
    {
        if (s1.age == s2.age)
        {
            return 0;
        }
        else if (s1.age > s2.age)
        {
            return 1;
        }
        else
        {
            return -1;
        }
    }
}

```

output:-

Sorting by id :

100 ram 12

102 Kiran 13

103 Krishna 14

Sorting by name :

100 ram 12

102 Kiran 13

103 Krishna 14

Sorting by age :

100 ram 12

102 Kiran 13

103 Krishna 14

Generic class and Generic method :-

A class which support different datatypes is called as generic class.

Eg:- ArrayList, Vector.

```
ArrayList <Integer> ob1 = new ArrayList<Integer>();  
ArrayList<String> ob2 = new ArrayList<String>();
```

My own Generic class which supports different data types.

Q) write a java program which defines a variable and to testvar() to set variable and get var() to get variable.

Sol:- Normal :-

```
class ex  
{  
    int var;  
    public void setvar(int v)  
    {  
        var = v;  
    }  
    public int getvar()  
    {  
        return var;  
    }  
    Main()  
    {  
        ex ob = new ex();  
        ob.setvar(10);  
        S.O.P (ob.getvar());  
    }  
}
```

Generic :-

```
class ex<I>
```

```
{  
    I ex;  
    public void setvar (I v)  
    {  
        var = v;  
    }  
    public I get_var ()  
    {  
        return var;  
    }  
}
```

```
main ()
```

```
{  
    ex<String> ob = new ex<String>();  
    ob.setvar(20);  
    s.o.p (ob.get_var());  
    ex<string> ob = new ex<string>();  
    ob.setvar ("Ram");  
    s.o.p (ob.get_var());  
}
```

If `template [I]` is written adjacent to class then it is called generic class.

Generics are 2 types :-

- 1) Generic class
- 2) Generic function

If `template [I]` is written adjacent to method then it is called Generic function.

Q) write a java program which defines a method display and contains one argument and print the argument.

Sol:- Normal :-

```
class ex
{
    public void display (int a)
    {
        System.out.println (a);
    }
    public static void main (String[] args)
    {
        ex ob = new ex();
        ob.display (10);
    }
}
```

Generic :-

```
class ex
{
    public <T> void display (T a)
    {
        System.out.println (a);
    }
    public static void main (String[] args)
    {
        ex ob = new ex();
        ob.<Integer> display (10);
        ex ob1 = new ex();
        ob1.<String> display ("ram");
    }
}
```

Q) write a java program for addition of two data types using generic class.

```
package p1;  
public class Ex<T extends Number>  
{  
    public T add(T num1, T num2)  
    {  
        return (T) (Integer) ((Integer) num1 +  
            (Integer) num2);  
    }  
    public static void main (String[] args)  
    {  
        Ex<Integer> ob = new Ex<Integer>();  
        System.out.println(ob.add(10, 20));  
    }  
}
```

output :- 30.

Design pattern :-

Design pattern - Says that arranging classes in such a way that to accomplish particular task.

1. Singleton pattern

2. Factory pattern

3. Template pattern

4. Decorator pattern

Singleton pattern says that it ensures creation of single object and which is shared among the different request.

Early Singleton pattern
Late Singleton pattern.

Factory pattern :-

The factory pattern says that based on your choice it automatically gives the corresponding objects.

we have 1 factory class.

Eg:- Animal Kingdom,

if my choice is cat then it will give the cat object, if my choice is lion then it will give lion object.

Q) write a java program which demonstrate the factory pattern.

sol:-

```
package pl;  
import java.util.Scanner;  
abstract class Animal  
{  
    abstract String getAnimal();  
    abstract String makeSound();  
    abstract String lifespan();  
}
```

class Lion extends Animal

```
{  
    public String getAnimal()  
    {  
        return "Animal: Lion";  
    }  
    public String makeSound()  
    {  
        return "Sound: Roars";  
    }  
    public String lifespan()  
    {  
        return "lifespan: 35";  
    }  
}
```

class cat extends Animal

```
{  
    public String getAnimal()  
    {  
        return "Animal: cat";  
    }  
    public String makesound()  
    {  
        return "Sound: Meows";  
    }  
    public String lifespan()  
    {  
        return "Lifespan: 12";  
    }  
}
```

```
public class AnimalFactoryDemo
```

```
{  
    public static Animal getAnimalType(char choice)  
    {  
        Animal ani = null;  
        if (choice == 'l')  
        {  
            ani = new Lion();  
        }  
        else if (choice == 'd')  
        {  
            ani = new Dog();  
        }  
        else if (choice == 'c')  
        {  
            ani = new Cat();  
        }  
        return ani;  
    }  
}
```

```
public static void main(String[] args)
```

```
{  
    Scanner sobj = new Scanner(System.in);  
    Animal aobj;
```

```
    System.out.println("Welcome to  
        Animal Planet");
```

```
    System.out.println("1. Lion(l)");
```

```

System.out.println("2. Dog (d)");
System.out.println("3. Cat (c)");
System.out.println("Enter your choice :");
aobj = getAnimalType(sobj.next().
charAt(0));
System.out.println(aobj.getAnimal());
System.out.println(aobj.makesound());
System.out.println(aobj.lifespan());
}
}

```

Template pattern :-

Template pattern is one of the designed pattern. In parent class you define a template (a order of the function execution). That will force to the child class i.e the child class has to follow the parent class template.

→ we define template as a final, (build computer function).

Eg :-

```

abstract Computer
{
    public final Build()
    {
        addMotherboard();
        SetupMotherboard();
    }
}

```

```

        addprocessor();
    }
    public abstract addmotherboard();
    public abstract setupmotherboard();
    public abstract addprocessor();
}

```

Q) write a java program which demonstrate the template pattern?

→ In template pattern, an abstract class exposes defined ways.

→ to execute this method. Its subclasses can override the method.

→ Implementation as per need but the invocation is to be in same way.

Sol:-

```

abstract class ComputerBuilder
{
    public final void buildComputer()
    {
        addMotherboard();
        setupMotherboard();
        addProcessor();
    }
    public abstract void addMotherboard();
    public abstract void setupMotherboard();
    public abstract void addProcessor();
}

```

class StandardComputer extends ComputerBuilder

```
{  
    public void addMotherboard()  
    {  
        System.out.println("Standard Motherboard");  
    }  
    public void setupMotherboard()  
    {  
        System.out.println("Plugging the power supply");  
    }  
    public void addProcessor()  
    {  
        System.out.println("standard Processor");  
    }  
}
```

class HighEndComputer extends ComputerBuilder

```
{  
    public void addMotherboard()  
    {  
        System.out.println("HighEnd motherboard");  
    }  
    public void setupMotherboard()  
    {  
        System.out.println("plugging the power");  
    }  
    public void addProcessor()  
    {  
        System.out.println("HighEnd Processor");  
    }  
}
```

```
public class TemplatePatternDemo
```

```
{
```

```
    public static void main (String[] args)
```

```
    {
```

```
        ComputerBuilder obi = new StandardComputer()
```

```
        obi.buildComputer();
```

```
        obi = new HighEndComputer();
```

```
        obi.buildComputer();
```

```
    }
```

```
}
```

Decorator pattern :-

Q) write a java program which demonstrate the decorator pattern?

```
Q1: package p1;
interface Shape
{
    void draw();
}
class Rectangle implements Shape
{
    public void draw()
    {
        System.out.println("Shape : Rectangle");
    }
}
class Circle implements Shape
{
    public void draw()
    {
        System.out.println("Shape : Circle");
    }
}
abstract class ShapeDecorator implements Shape
{
    protected Shape decoratedShape;
    public ShapeDecorator (Shape decoratedShape)
    {
        this.decoratedShape = decoratedShape;
    }
    public void draw()
    {
        decoratedShape.draw();
    }
}
```

```

class RedShapeDecorator extends ShapeDecorator
{
    public RedShapeDecorator (Shape decoratedShape)
    {
        super (decoratedShape);
    }
    public void draw ()
    {
        super.draw ();
        setRedBorder (decoratedShape);
    }
    private void setRedBorder (Shape decoratedShape)
    {
        System.out.println ("Border color: Red");
    }
}

public class ShapeDecoratedDemo {
    public static void main (String[] args) {
        Shape circle = new Circle ();
        Shape redCircle = new RedShapeDecorator (
            new Circle ());
        Shape redRectangle = new RedShapeDecorator (
            new Rectangle ());
        circle.draw ();
        redCircle.draw ();
        redRectangle.draw ();
    }
}

```

Exceptions :-

Syntax errors are identified at compile time.

Eg: forgetting ; at end of the statement,
forgetting close brace (}) for the for loop.

Exceptions are the runtime errors.

Exception is an event that will list up the normal flow of execution then it terminates the program immediately. i.e. it will not execute the remaining statements.

Exception cause abnormal termination of the program. i.e. not visible for the programming environment.

In order to rectify that we have a concept called exception handling.

Exception handling is a mechanism to handle the exception.

we have diff exception handling mechanism

They are :-

- 1) try catch block
- 2) multiple catch
- 3) nested try catch block
- 4) finally block
- 5) throw and throws keyword
- 6) user defined exception.

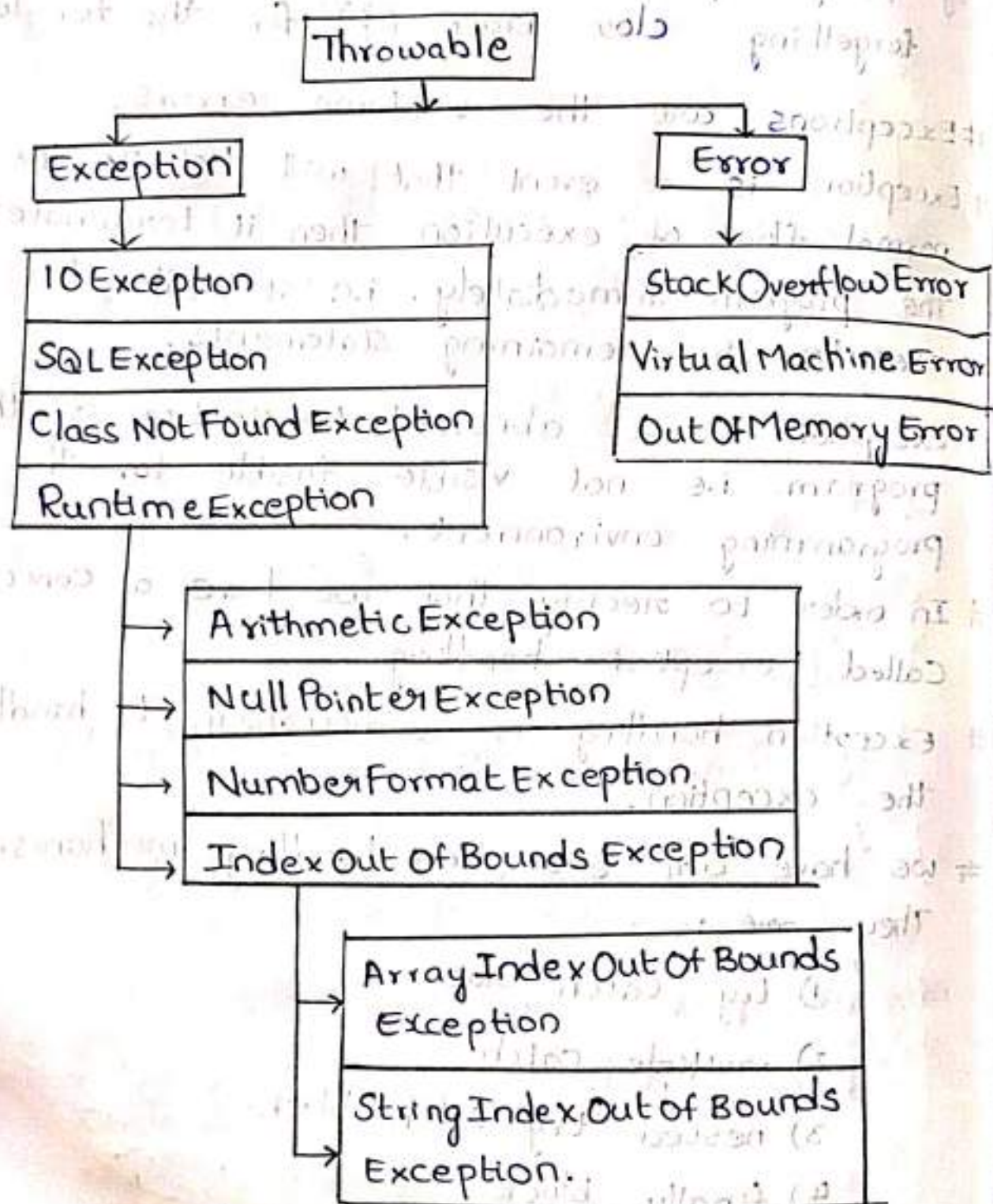
Some examples of exception :-

- 1) divided by 0 - Arithmetic exception.

2) Null pointer exception

3) arrayindex out of bound exception.

Hierarchical of exception package :-



Here first letter of every word is Capital due to it is class name.

Q) write a java program which shows the divided by zero error. (Arithmetic exception)

where try block is written there is a chance of getting exception, those statements are enclosed by try block.

→ Immediately after the try block we need to write catch block i.e. we should not write any statements b/w try and catch block.

→ If exception raised in try block automatically control will shift to catch block

```
Q1: package pl;  
class exception  
{  
    public static void main (String[] args)  
    {  
        int k  
        {  
            try  
            {  
                k = 30/0;  
            }  
            catch (ArithmeticException e)  
            {  
                System.out.println("zero division unable");  
            }  
            System.out.println("rest of code");  
        }  
    }  
}
```

output: Zero operation division unable.
rest of code.

→ if exception is not raised, catch block will not executed but it will be ignored.

Q) write a java program which demonstrate NullPointerException and ArrayIndexOutOfBoundsException?

sol:

```
package pl;  
  
public class Exceptionseg  
{  
    public static void main (String[] args)  
    {  
        int a[] = new int[3];  
        try  
        {  
            a[4] = 23;  
        }  
        catch (ArrayIndexOutOfBoundsException e)  
        {  
            System.out.println ("Index bounce");  
        }  
        String s = null ;  
        try  
        {  
            System.out.println (s.length ());  
        }  
        catch (NullPointerException e)  
        {  
            System.out.println ("null exception");  
        }  
        System.out.println ("rest of the code");  
    }  
}
```

multiple catch :-

If multiple catches are there for single try block, if 1 catch is executed then remaining catch will be ignored.

Multiple catches can be written in any order.

Q) write a java program which demonstrate the multiple catches?

Sol:- package pl;

public class multiplecatches

{
public static void main (String[] args)

{
int a[] = new int[3];

try

{
a[4] = 23/0;

catch (ArrayIndexOutOfBoundsException e)

{
System.out.println ("Index bounce");

catch (ArithmeticException e)

{
System.out.println ("arithmetic");

System.out.println ("rest of the code");

}.
}

Q) Write a java program, if the age is less than 18 then trigger the arithmetic exception and print that not eligible for vote otherwise eligible for vote.

Sol:-

```
package pl;  
import java.util.Scanner;  
  
public class Voters  
{  
    public static void main (String[] args)  
    {  
        int age;  
        Scanner sc = new Scanner (System.in);  
        age = sc.nextInt();  
        try  
        {  
            if (age < 18)  
            {  
                age = 18/0;  
            }  
            else  
            {  
                System.out.println ("Eligible for vote");  
            }  
        }  
        catch (Exception e)  
        {  
            System.out.println ("Not Eligible");  
        }  
        System.out.println ("rest of code");  
    }  
}
```

Finally block:-

It is a block, whether exception occurred or not. It will always execute the finally block.

The purpose of finally block is, it will deallocate all the resources.

Syntax :-

finally

```
{  
    // resource deallocate  
}
```

Q) write a java program which demonstrate the finally block.

```
1. package p1;  
   public class fin {  
       public static void main (String args[])  
       {  
           try  
           {  
               int k = 23/0;  
           }  
           catch (ArithmeticException e)  
           {  
               System.out.println ("Divided by zero");  
           }  
           finally  
           {  
               System.out.println ("resource dellocate");  
           }  
       }  
   }
```

if the exception is not raised it will not executed the catch block.

whether exception raised or not compulsory the finally block will be executed

The finally block should be written at the end of all the catches.

Nested try catch:-

```
try
{
    try
    {
        }
    catch ( ) → ArithmeticException
    {
        }
    }
catch ( ) → NullPointerException
{
    }
}
```

Q) write a java program which demonstrate the Nested try catch.

sol:-

```
package p1;
public class Nestedtrycatch
{
    int a[] = new int[3];
    String s;
```

```

public static void main (String[] args)
{
    try
    {
        try
        {
            int k = 23/0;
        }
        catch (ArithmeticException e)
        {
            System.out.println ("Arithmetic exception");
        }
        r = null ;
        System.out.println (r.length);
    }
    catch (Null-pointerException e)
    {
        System.out.println ("Null pointer");
    }
}
}

```

output :

Arithmetic exception

Null pointer.

throw and throws :-

throw - to raise the exception we use throw keyword

throws - to propagate the exception from one function to another function.

Syntax for throw :-

throw new Exception name ("message");

Eg:-

throw new ArithmeticException (" \ by 0 ");

Q) write a java program which demonstrate the throw keyword.

Sol:- package pl;

public class throwword

{ public static void main(String[] args)

{

try

{

throw new ArrayIndexOutOfBoundsException
- on ("array out");

}

catch (ArrayIndexOutOfBoundsException e)

{

System.out.println (e.getMessage());

}

}

};

#throws :-

always write adjacent to function only.

Eg:-

public void add() throws ArithmeticException.

Q) write a java program which demonstrate throws keyword.

Sol package pl;

public class throws word

{
public void m() throws ArithmeticException

{
int k = 23/0;

}

public static void main (String args[])

{

throws3 t1 = new throws3();

try

{

t1.m();

}

catch (ArithmeticException e)

{

System.out.println ("divided by zero");

}

}

}

User defined Exception :-

So far, we have seen that predefined exceptions like `ArithmeticException`, `NullPointerException`, `Exception`, etc..

Now we can define your own exception class.

Step-1 :- Create your own class which extends the exception class.

Step-2 :- write a constructor with message and call the super keyword

Q) write a java program which demonstrate your own exception.

```
Sol:- package P1;
public class own extends Exception
{
    own (String s)
    {
        super(s);
    }
    public static void main (String[] args)
    {
        try
        {
            throw new own ("own exception");
        }
        catch (own e)
        {
            System.out.println (e.getMessage());
        }
    }
}
```

Strings :-

strings are immutable i.e we cannot modify the content, once it is defined. we can define strings in 2 ways.

```
String S1 = "Ram";
```

```
String s2 = new String("kumar");
```

Without using the new keyword, the memory is allocated to scp (string constant pool).

before allocating the memory, it will check whether the string is already there or not.

If already there returns the same address or else it will create a new address

when you are creating a string with new keyword, memory is allocated from the heap memory.

String Buffer and String Builder are mutable.

```
S1 = "Ram"; S2 = "Ram"
```

```
if (S1 == S2)
```

↓
checks the address

```
if (S1.equals(S2))
```

↓
checks equal or not

Methods in String class :-

```
String.toCharArray()
```

```
String.length()
```

```
String.charAt()
```

```
String.equalsString()
```

String.toCharArray() :-

It convert string to array.

String.concat() :-

It will concat at the two strings

Eg:- S1 = "nam";

S2 = "kumar";

S1.concat(S2)

= namkumar.

String.startsWith() and String.endsWith() :-

S1 = "Andhra Pradesh";

S1.startsWith("Andhra");

↳ True.

startsWith() will check that string is start with specific string.

S1.endsWith("Pradesh");

↳ True.

String.toUpperCase() and String.toLowerCase() :-

S1 = "ram";

S2 = "KUMAR";

S1.toUpperCase()

= RAM

S1.toLowerCase()

= kumar.

TAVA

String.equalsIgnoreCase() :-

It acts like a equals method but, it is a case sensitive.

String.indexOf() :-

sl = "Ram" ;

sl.indexOf(a) = 1

It checks from left to right.

String.lastIndexOf() :-

It checks from right to left.

sl = "Andhra Pradesh"

0 1 2 3 4 5 6 7 8 9 10 11 12 13

sl.lastIndexOf(a) = 9

String.split() :-

sl = "welcome to KL University";

String a[] = sl.split(" ");

a[0] = welcome

a[1] = to.

String.Join() :-

It is quite opposite to split method.

String.Join(" * ", a[0], a[1], a[2], a[3]);

= welcome * to * KL * University.

String.Substring() :-

sl.substring(3, 7) - we can skip the part of the string.

st. substring (10)

= KL University.

string.replace() :-

st. replace ("e", "*");

it replace a character in a string.

String.valueOf() :-

It will convert integer to string.

String.valueOf(100); i.e from one data type to string.

string builder and String Buffer are mutable i.e we can change content.

The above 2 functions class will have all the 18 functions from string class. Apart from that we have four methods append(), insert(), delete(), deleteChar()

List :-

1. String.toCharArray()
2. String.Length()
3. String.charAt()
4. String.equalsString
5. String.toConcat()
6. startWith()
7. endWith()
8. String.toUpperCase()
9. String.toLowerCase()
10. String.equalsIgnoreCase()
11. String.IndexOf()
- 12.

Files :-

files concept is available in java.io package

we have so many classes in io. package.

Five classes are important for holding the files concept. They are

1. File

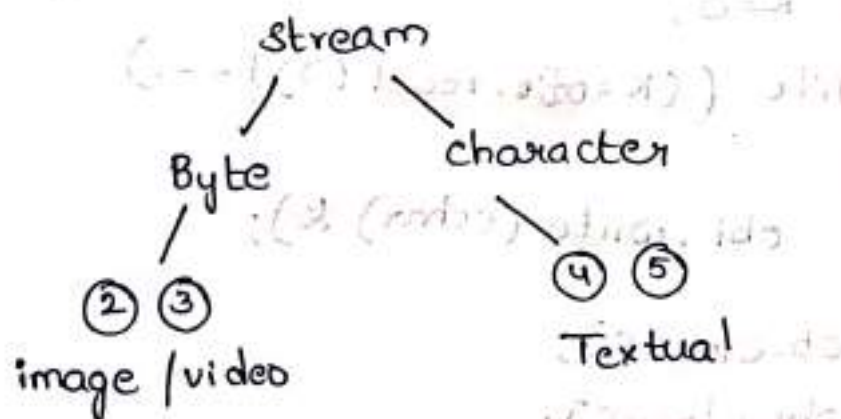
2. FileInputStream

3. FileOutputStream

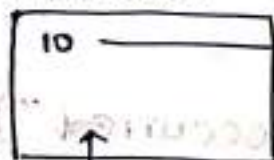
4. FileReader

5. FileWriter

streams :- It is a sequence of data. we have two types.



File1.txt



FileInputStream - read()

FileReader - read()

↳ byte
↳ character / Integer

to put data into File

FileOutputStream - write()

↳ byte

FileWriter - write()

↳ string / character / int

Q) write a java program which Copies from one file1.txt to another file file2.txt by using byte oriented stream.

Sol:

```
package pl;  
import java.io;  
public class myfile  
{  
    public static void main (String[] args)  
    {  
        try  
        {  
            FileInputStream ob = new FileInputStream(" ");  
            FileOutputStream obl = new FileOutputStream(" ");  
            int k=0;  
            while ((k=ob.read())!=-1)  
            {  
                obl.write((char) k);  
            }  
            ob.close();  
            obl.close();  
        }  
        catch (Exception e)  
        {  
            System.out.println ("error occurred");  
        }  
    }  
}
```

- # In the files concept when you are creating object for `FileInputStream` it generates an exception called `File not found exception`
- # `ob.read()` will generate one exception i.e `io.exception`.
- # `ob1.write()` will generate one exception i.e `io.exception`.

Q) write a java program for above problem using `file reader` and `FileWriter`?

Soln

Q) write a java program which add integer
10, 20 from the file1.txt and write the
result (30) in file2.txt